Università degli Studi di Catania

DIPARTIMENTO DI INGEGNERIA ELETTRICA ELETTRONICA E INFORMATICA

Corso di Laurea Magistrale in Ingegneria Informatica

Tesi di laurea magistrale

Service-Related Data Loss Assessment on a Ceph Cluster

Candidato:
Michele Cannizzaro
Matr. O55000388

Relatore:
Prof.ssa Antonella Di Stefano

Correlatore:
Prof. Giovanni Morana

Anno Accademico 2022–2023

Acknowledgement

I would like to express my deepest gratitude to Professor Antonella Di Stefano and Professor Giovanni Morana for their expertise and support throughout the process. Thanks should also go to Aucta Cognitio employees. Lastly, I would be remiss in not mentioning my family, especially my mother and father, who have always supported me during these university years.

Abstract

The need to store massive amounts of data creates the requirement for storage systems capable of expanding their capacity accordingly. Among the open-source distributed storage solutions available, Ceph notably stands out. Although Ceph is primarily self-contained in caring for itself and recovering from failure scenarios, the data loss problem is complex. It depends on many factors, including the OSDs distribution across buckets and the various types of replication and fault domain policies utilized in pools. In this complex, heterogeneous, and ever-changing scenario, it is challenging for a Ceph system administrator to individuate the best Ceph deployment to mitigate the risk of data loss a priori.
Furthermore, it is difficult to know how sensitive the data is to the network or bucket failures. Equipping the administrator with a comprehensive view of which pools are affected by data loss is crucial to provide a data loss service-oriented perspective.

This thesis aims to supply Ceph system administrators with a broad and service-oriented view of a Ceph cluster during failures or in their anticipation. By equipping them with reactive and proactive tools to gain knowledge on the data loss probability of the pools, administrators can make informed decisions regarding the optimal Ceph deployment strategy before failures. This approach helps mitigate the risk of data loss in the pools.

The study has developed functions for incremental data loss probability assessment and forecasting, which are integrated into a server component deployed within a Ceph cluster. REST API and RPC clients have also been implemented to interact with the server. The collective integration of these components forms a comprehensive Service-Related Data Loss Evaluation System.

Contents

Introduction **1**
The Growing Trend of Data Production 1
Distributed Storage: File, Block, Object Storage 2
Ceph an Open-Source Storage Platform 4
Data Loss Mitigation and Its Match with the Service 5

1 Ceph **7**
Ceph Architecture Goals and Design Features 7
Main Components 8
Ceph Client Types and Patterns 9
Dynamically Distributed Metadata 12
Ceph Cluster Daemons 13
Objects, Pools, Placement Groups, and OSDs 15
Ceph Input/Output Operations 17
The Reason Behind Placement Group Architectural Choice 18
The CRUSH Algorithm, Map, and Ruleset 20
OSDs are Intelligent Storage Devices 27
Replication and Erasure Coding 27
Failure Detection and Data Integrity 30

2 Troubleshooting and Data Loss **34**
Troubleshooting Monitors 34
Troubleshooting OSDs 36
Troubleshooting PGs 38
Data Loss 39
Service-Related Data Loss Assessment 43

3 Service-Related Data Loss Evaluation System **44**
Overview 44
Chosen Programming Languages and Tools 44
Deploying a Ceph Cluster for Testing Purposes 46
Parsing Functions and Mapping Cluster Hierarchy 47
Incremental Data Loss Assessment Functions 58
Data Loss Forecasting Function 66
Service-Related Data Loss Evaluation System 68

Conclusion **82**

Bibliography **86**

List of Figures

1.1 Ceph main components [35] 8
1.2 RGW position in the stack [7] 9
1.3 RGW workflow: (1) metadata looking, (2,5) bucket update, (3) striping data, (4) update bucket index [30] 10
1.4 librbd position in the stack [6] 10
1.5 Object-Watch-Notify pattern [17] 11
1.6 Striping pattern [17] 12
1.7 Subtrees mapping of the directory hierarchy to metadata servers [33] 13
1.8 Ceph cluster daemons [17] 14
1.9 Objects - Pool - Placement Groups - OSDs mapping [30] 17
1.10 Three different design alternatives within a replication scenario [30] . 18
1.11 Scenario in which a device failure occurs [30] 19
1.12 Concurrent failures scenario [30] 19
1.13 A simple rule that distributes three replicas across three cabinets in the same row. [35] 24
1.14 Replication: writing scenario [17] 29
1.15 Erasure Coding (4,2) example for content "ABCDEFGHIJKL" [17] . 29
1.16 Rebalancing [8] 32

2.1 Prometheus graph of the number of OSDs not up (numOsdsNotUp) . 40

3.1 Prometheus architecture [27] 45
3.2 Output of "ceph orch host ls" command 47
3.3 Service-Related Data Loss Evaluation System 68
3.4 Matching server and client output in a sequence of REST API requests 74
3.5 Prometheus metrics on the production environment 79
3.6 Pool Data Loss Probability graph (Grafana) 80
3.7 Four Weeks Forecasting graph (Grafana) 80
3.8 Grafana dashboard of the Service-Related Data Loss Evaluation System metrics 80
3.9 Matching server and client output in a sequence of RPCs 81

Introduction

The Growing Trend of Data Production

Over the past two decades, the volume of digital data generated has grown exponentially. Even though the Internet became mainstream in the 1990s, it was in the early 2000s the Internet and the Web began this exponential growth trend in producing digital data in the form of semi-structured and unstructured data. The advent of social media and the diffusion of mobile devices have further contributed to this trend. With around 4.66 billion active Internet users worldwide [23], 2.5 quintillion bytes (two exabytes plus 500 petabytes) of data were created daily in 2020 [15]. To clearly define the data size under consideration, Facebook generates 4 petabytes of data daily; Google processes over 20 petabytes of data each day [28]; the collective writings of humanity in all languages from the dawn of civilization until the present day amount to roughly 50 petabytes of data, according to estimates [19]. Compared to traditional business, data generated by humans is experiencing an overall growth rate that is ten times faster.

Also, machine-generated data is growing even faster, 50 times than traditional business data. The trend of increased internet usage and, thus, digital data production for work, learning, and entertainment was further accelerated by the coronavirus pandemic, which led to the closure of offices, schools, restaurants, and other establishments, necessitating people to spend more time online. However, web-based content is one of many digital data sources of daily produced unstructured data. The growth of sensor-based internet-enabled devices increased data production like never before. Millions of wearables, TVs, washing machines, refrigerators, and many other objects that contain embedded technology to sense, communicate, and interact with their internal state or the external environment constitute what is known as the "Internet of Things" (IoT). According to the International Data Corporation (IDC) research, the amount of data generated by IoT devices could reach 73.1 ZB (zettabytes) by 2025 [18]. This tendency contributed to the general perception of being overwhelmed with data. The term "Big Data" quickly gained immense popularity and became a buzzword at the end of 2011 [16], leading businesses to retain data for potential

future value as part of the backup. A key factor explaining the hype around Big Data is that Big Data Analysis helps coordinate decisions in many business areas. For instance, Amazon and Netflix use correlation analysis to suggest products that might interest customers. Data holds immense power when appropriately utilized since it represents knowledge and insights that can be leveraged to make informed decisions. This massive quantity of data, called Big Data, also powers the evolution of Artificial Intelligence and Machine Learning. For instance, Deep Learning uses neural networks to recognize complex patterns in high-volume data to train them.

Unstructured data is a rich source of knowledge. According to analysts at Gartner, unstructured data represent an estimated 80-90% of all newly produced data [2]. The latter is data that can not be actively managed and stored in a structured database format, thus, in a set of tables with columns and rows. It is called "unstructured" because it has an internal structure not predefined through data models, as opposed to a structured one. Unstructured data comprise media, images, audio, surveillance data, weather data, sensor data, IoT data, analytics, text messages, call logs, customer feedback, documents, and many other data types.

The growing production of unstructured data in every field and the need to store them for future processing have created the requirement for storage systems to expand their capacity accordingly to the needs and to meet the growing demands for performance, reliability, and fault tolerance. Undoubtedly, the traditional storage approach comprising stand-alone and specialized storage boxes no longer works for these kinds of challenges. Distributed storage is the approach to take.

Distributed Storage: File, Block, Object Storage [3][4][14]

In distributed storage systems, data is stored in three primary forms: file, block, and object storage.

File Storage, also known as file-level or file-based storage, manages data as a hierarchical tree structure. Data is stored in files organized under a hierarchy of directories and subdirectories. Files are uniquely identified through paths, which are strings of characters that include the file name, extension, and the path itself. File storage systems rely on a block device as an abstraction layer whose size is a multiple of the physical block size. However, this design choice results in limited scalability and inefficient space utilization due to internal fragmentation caused by file lengths not always aligning with the block size.

Block Storage, also known as block-level or block-based storage, stores data into

blocks as a sequence of data bytes. Each block is specified with a unique identifier.

Blocked data are generally stored in a data buffer and read or written one block at a time. In this manner, the handling of the data stream is speeded up, and overhead is reduced. It is also compatible with storing sizeable random access files, including application logs or databases. While it is an efficient and reliable method for managing structured data, it inherently struggles to manage unstructured data because of its poor ability to handle metadata. Actually, this method involves dividing data into equally sized blocks without any supplementary information about their contents. When it deals with an amount of data beyond a hundred terabytes or multiple petabytes, it may run into durability issues, storage infrastructure limitations, and management overhead.

Object Storage, also called object-based storage, manages data as objects. It is well suited for storing large amounts of unstructured data and comes at a significantly lower price per gigabyte than a distributed filesystem. That is because object-based storage couples processors and memory disk drives, delegating low-level allocation to object storage devices (OSDs) and decoupling I/O from metadata operations. Objects are organized in a flat bucket, and only the atomic replacement of an entire object is allowed. The ability to edit one object or part of it incrementally, as file systems do, is not provided. Lower-level functionalities, like space management, are moved into the storage device itself, and data management functions like data replication and data distribution are enabled at object-level detail. In comparison with the file system, object storage provides only eventual consistency.

As improving the overall reliability and scalability of storage systems has become critical, file system designers continue to look for new architectures to meet these conditions. Although object storage systems seem ideal for solving the increasing problems of production data growth, they inherit decades-old architectural choices that limit their ability to scale to hundreds of petabytes. Even though they decouple I/O from metadata operations, this kind of solution continues to suffer scalability limitations because of the lack of distribution of the metadata workload. Other factors have also contributed to these limitations, including the continued reliance on traditional file systems principles like allocation lists and inode tables and the reticence to delegate intelligence to OSDs.

In the group of open-source distributed storage solutions available in 2023, Ceph, a distributed object store, notably stands out. Among its key design features, they designed this to overcome scalability limitations by replacing file allocation tables with generating functions. The future of storage lies in distributed object stores,

which can accommodate unstructured and structured data while enabling clients to utilize modern and legacy interfaces simultaneously.

Ceph an Open-Source Storage Platform

Ceph is a software-defined storage platform implementing a distributed object store on a cluster built from commodity hardware components. It provides a unified storage service with object, block, and file interfaces. Initially created by Sage Weil for his doctoral dissertation at the Jack Baskin School of Engineering, University of California, Santa Cruz (UCSC), it debuted in November 2006 with Weil giving two presentations at USENIX OSDI and SC'06 [25]. Designed with excellent performance, reliability, availability, and scalability to the exabyte level (and beyond) in mind, Ceph can achieve all of this through three key design features: decoupled data and metadata, dynamic distributed metadata management, and reliable autonomic distributed object storage.
Through replication, erasure coding, snapshots, and storage cloning, Ceph offers disaster recovery and data redundancy. Mechanisms including Scrubbing and CRC checks are used to protect against bad disk sectors and bit rot, thus maintaining data integrity. In addition, Ceph is self-healing and self-managing and pushes throughput limitations, which impact performance and scalability, by allowing each file to specify its own striping strategy and object size. Using the CRUSH algorithm, data are placed evenly but pseudo-randomly throughout the system, considering different failure domains.
Ceph is employed to build resilient clusters in different use cases, like:

- Big Data workloads
- Private cloud infrastructure
- Public cloud storage
- Rich media
- Microservice and container-based workloads (like OpenStack or Kubernetes)

Despite being a favorable choice for storage in numerous situations, Ceph also has its disadvantages that should be considered:

- It requires a comprehensive network due to the variety of components provided
- It takes a relatively long time to configure the storage cluster
- It requires auxiliary engineering oversight to implement and manage the cluster

Closely related to the last disadvantage is the lack of a mechanism for assessing data loss risk about the service provided. This drawback makes it difficult for the Ceph system administrator decide the best Ceph deployment to mitigate the risk of data loss guaranteeing the service before failure.

Data Loss Mitigation and Its Match with the Service

As previously stated, data is the backbone of businesses. Consequently, data loss can give rise to many costs and far-reaching consequences that can significantly impact organizations. Firstly, it can affect the functionality of an entity, forcing time and resources to be diverse to address the recovery of lost data. Moreover, data loss can inflict severe damage on the reputation of a business that offers its customers storage services, leading to customer dissatisfaction, erosion of loyalty, and, ultimately, the loss of valuable clientele. Beyond the immediate operational and reputational impact, data loss can have dire financial implications. It can deprive a company of revenue-generating capabilities, especially if important business data, customer records, or transactional information is irretrievably lost. This revenue loss can disable the ability of the organization to sustain its operations, meet financial obligations, and invest in future growth. In severe cases, it can even push a company towards the brink of bankruptcy, as the financial strain becomes insurmountable. Lastly, data loss can expose organizations to legal consequences. Depending on the nature of the lost data and applicable data protection regulations, there may be legal obligations to safeguard customer data and protect sensitive information. Thus, preventing data loss is crucial.

Within a Ceph cluster, the predominant factor contributing to outages and data loss is directly connected to the loss of several OSDs that exceed the replication level within a short period. These occurrences typically arise from software or hardware failures. The data loss becomes irrevocable if these OSDs fail to reconnect and Ceph cannot recover the affected objects in-between OSD failures.

Although Ceph is primarily self-contained in taking care of itself and recovering from failure scenarios, the data loss problem in Ceph is complex. It depends on many factors, including OSDs distribution, replication policies, storage device utilization, and network failures. This thesis aims to develop tools to assist Ceph system administrators in visualizing service-related data loss probability, giving them the knowledge to make informed decisions about cluster management when human intervention is required. Data loss risk assessment and forecasting functions are introduced to evaluate and predict the impact of failures on data within pools. The ultimate goal is to provide administrators with a comprehensive view of the Ceph cluster during failures or in anticipation of them, enabling proactive measures to

mitigate the risk of data loss.

The first chapter comprehensively overviews the Ceph architecture and its objectives. It delves into the workings of the CRUSH algorithm, explores different redundancy strategies including replication and erasure coding, and places particular emphasis on failure detection and data integrity.

The second chapter discusses the common issues of Ceph and how to troubleshoot them to restore its functionalities. While many issues may impact the operation of the cluster without resulting in a complete system outage or data loss, some scenarios can lead to such undesirable outcomes. This chapter focuses on data loss and highlights the significance of considering service-oriented data loss in Ceph deployments.

The third chapter describes in detail the components comprising the Service-Related Data Loss Evaluation System. This system aims to assist the Ceph system administrators in evaluating and forecasting service-related data loss, enabling them to proactively identify measures to mitigate the potential risk of data loss.

Chapter 1

Ceph

Ceph Architecture Goals and Design Features

Ceph aims to avoid the problems of legacy client-server architectures. As the services scale out, it becomes difficult for the system to maintain the illusion of being a single server when it comprises hundreds or thousands of individual servers. Even though there are several tricks to maintain this illusion (virtual IPs, failover pairs, gateway nodes), these design methods limit the overall system design, performance, and consistency [30]. Ceph is designed around what is called a client cluster architecture. This definition means that an intelligent client library on the application side understands that it is not talking to a single server but to a cluster of cooperating servers. The library in question, called Librados, performs the smart routing of requests, the flexible addressing of the network, and provides the same simple API to applications. On the cluster side, RADOS (Reliable Autonomic Distributed Object Store), a reliable object storage service, gives clients the illusion of interacting with a single logical object store [34].

Ceph primary architecture goals are availability, performance, reliability, and scalability. These architectural goals can be achieved through three fundamental design features:

- **Decoupled Data and Metadata**:
  File system data storage and metadata management are separated. A metadata server cluster manages metadata operations (for example: open and rename), while clients interact directly with OSDs to perform file I/O operations (reads and writes).

- **Dynamic Distributed Metadata Management**:
  Metadata management is critical to the overall system performance because metadata operations comprise much as 50% of file system workloads. Ceph utilizes a metadata cluster architecture based on Dynamic Subtree Partitioning to

distribute responsibility adaptively and intelligently among multiple metadata servers (MDSs). The workload distribution among metadata servers is based on current access patterns. This architecture allows Ceph to utilize available resources effectively under any workload.

- **Reliable Autonomic Distributed Object Storage**:
  Large clusters are dynamic and are built incrementally. In this scenario, device failures are systematic and expected, so it is required that the data distribution effectively utilizes available resources maintaining the desired level of data replication. Ceph achieves this by delegating responsibilities of replication, data migration, failure detection, and recovery to the cluster of OSDs storing the data [33, p.2].

Main Components

Ceph has three main components: the Ceph client, a cluster of OSDs, and a metadata server cluster (MDS).

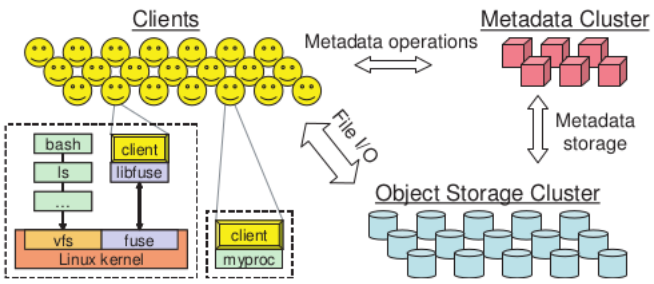


Figure 1.1: Ceph main components [35]

The Ceph client runs on each host and presents an interface that appears as a file system to applications [33, p.3]. Ceph provides clients with object, block, and file interfaces to storage services named RADOS gateway (RGW), RADOS block device (RBD), and RADOS file system (CEPHFS), respectively.

The cluster comprises intelligent OSDs. Since there is no centralized gateway, typical in traditional architectures, the cluster has no single point of failure letting clients interact directly with Ceph OSD Daemons. Systems composed of many thousands of devices are inherently dynamic. Furthermore, device failures are frequent and expected, as well as creating, moving, or deleting volumes of data. The cluster can grow or shrink as new storage is deployed or old devices are decommissioned.

The MDS manages file system namespace, stores metadata in objects, and can be distributed over hundreds of nodes while providing consistency, reliability, and

high performance. It uses dynamic subtree partitioning to distribute a complicated hierarchical file structure across a near-linear set of metadata servers. The MDS cluster can be made arbitrarily scalable by adding more MDSs [33].

Client Types, Ceph Services and Patterns

Ceph clients use the Reliable Autonomic Distributed Object Store (RADOS) protocol to interact with the cluster. Ceph provides three diverse types of clients, which differ materially in how they present data storage interfaces to the storage services offered by Ceph, as mentioned in the previous section.

RADOS Gateway (RGW)

An object storage interface that supplies object storage services in Ceph. As the name suggests, Rados Gateway provides a RESTful gateway between applications and Ceph Clusters. Two interfaces are supported: S3-compatible and Swift-compatible. The former is compatible with a subset of the Amazon S3 RESTful API, while the latter is compatible with a subset of the OpenStack Swift API. Ceph object storage uses the Ceph Object Gateway daemon, an HTTP server designed to interact with the Ceph cluster through librados [7]. RGW is often combined with a load balancer

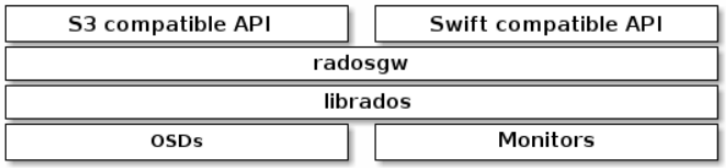


Figure 1.2: RGW position in the stack [7]

to provide storage service to the public internet. The S3 API is built around the idea of having users and bucket collections of objects. The S3 objects are different from the ones analyzed in the subsequent sections of this chapter. RADOS objects are small compared to the S3 objects, which are big and live within buckets, a different abstraction in implementation from the one utilized by RADOS. The RGW receives the S3 objects from the client and redirects them across several smaller RADOS objects. Figure 1.3 shows how RGW stores its data in RADOS. Observing the image, the client sends a video file through an S3 PUT operation into the RGW, which will execute in order these actions:

- Looks at S3 object metadata (user and bucket info) to make sure that the request is valid

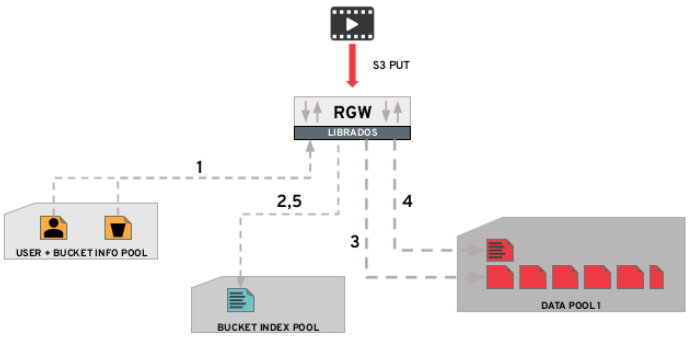


Figure 1.3: RGW workflow: (1) metadata looking, (2,5) bucket update, (3) striping data, (4) update bucket index [30]

- Makes an update to the bucket index (maintained to perform objects enumeration in a bucket)
- Stripes data across a lot of RADOS objects
- Updates again the bucket index

It is crucial to mention that Ceph Object Storage does not use the MDS.

RADOS Block Device (RBD)

It facilitates the storage of block-based data in the Ceph cluster. Block-based application data are broken into small chunks and stored subsequently as RADOS objects, which are stored in virtual block devices. These are the virtual equivalent of physical disk drives [29]. Thus, a block-based storage interface aims to provide a virtual block device that stores disk images in the Ceph cluster, striping data across many RADOS objects in a pool. Its purpose is to take the storage and decouple it from the host. RBD is accessible via virtualization technologies (like KVM/QUEMU) and raw Linux host. Furthermore, Ceph block devices leverage RADOS capabilities, for example, snapshotting, replication, and strong consistency. RBD clients communicate with the Ceph cluster through kernel modules or the "librbd" library [6].

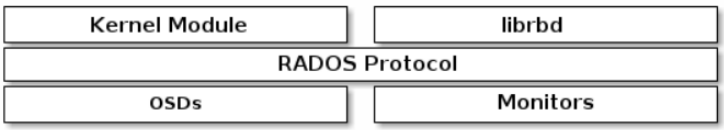


Figure 1.4: librbd position in the stack [6]

File Storage (CEPHFS)

CephFS is a distributed network file system built on top of RADOS. It is designed to allow concurrent shared access to the file system namespace from multiple clients. It provides a strong level of consistency and caching. Data and metadata are separated in different RADOS pools. Metadata is served via a Metadata Server Cluster (MDS). Clients have direct access to RADOS for reading and writing file data blocks without a gateway or a broker as a mediator. All the metadata operations are journaled in a metadata journal. There are several different ways to access CephFS. The common ones involve using a Linux Kernel client or "ceph-fuse" implementation.

All Ceph clients have the exact basic needs and tend to follow similar patterns. They need to know the pool name, the Ceph monitor address (or Ceph configuration file), the user name, and the path to the secret key. It is worth describing two typical patterns that Ceph clients utilize: object-watch-notify and striping patterns.

Object-Watch-Notify

This pattern describes the client registering its persistent interest in an object, which then leads to maintaining an open session between the client and the Primary OSD. Any object can be utilized as a synchronization/communication channel: the client sends a notification and the payload to all watchers and receives an acknowledgment when they receive the notification. On top of this pattern is built the feature of Mandatory Exclusive Locks, which helps to address the write conflict situation when multiple clients try to write to the same object. So, before a client can write to an object, it must check if another mounted client (peer) has established an exclusive lock on the same object. Only a client at a time can modify an object [17, p.22-23].

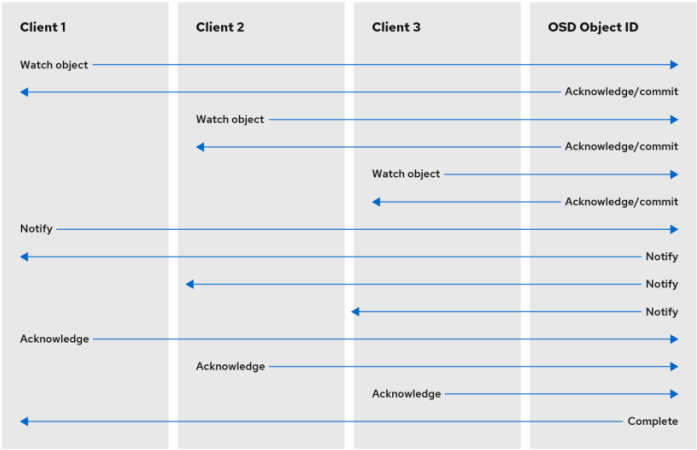


Figure 1.5: Object-Watch-Notify pattern [17]

Striping

Striping consists of distributing data across multiple storage devices to overcome throughput limitations and increase performance. By spreading writing operations over multiple objects, operations can occur in parallel at the maximum speed. Ceph clients convert their data from the representation format provided to the users (objects, block, filesystem directories) into objects for storage in the Ceph Storage cluster. Specifically, the data is striped over multiple Ceph Storage cluster objects. It is essential to notice that the objects in themself are not striped. Ceph clients stripe data to stripe units, which subsequently are mapped to objects. The set of objects containing the sequence of stripes unit is called an "object set." By the CRUSH algorithm, objects are mapped to placement groups, which then are mapped to OSD daemons before the objects are stored as files on a disk [17, p.25-28].

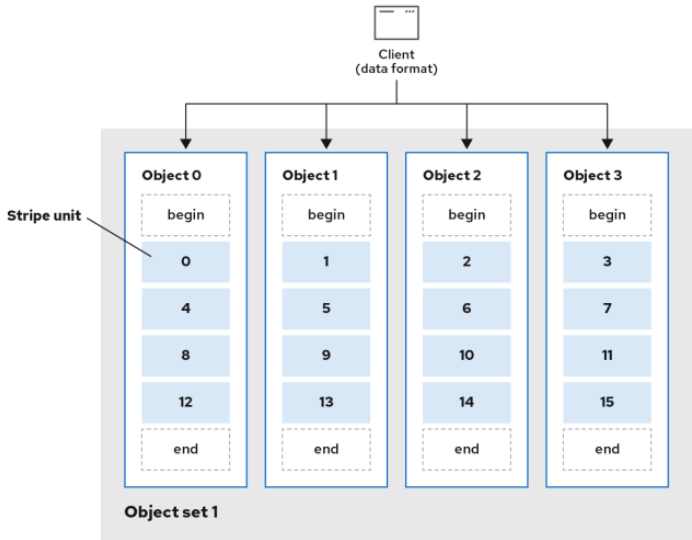


Figure 1.6: Striping pattern [17]

Dynamically Distributed Metadata

Efficiently addressing metadata scaling challenges is key to achieving high-performance and reliable distributed file systems. While capacity and aggregate I/O rates can be scaled up easily by adding more storage devices, metadata operations introduce a higher level of interdependence, making it more challenging to achieve scalable consistency and coherence management. Operations, including file system directory operations or file attribute modifications, often require coordination and synchronization across multiple nodes in the distributed system. In Ceph, thanks to the absence of file allocation metadata (object names are constructed using the inode

number), the MDS cluster efficiently manages and simplifies the metadata workload. As mentioned in the previous sections, the Distributed Metadata Server (MDS) cluster is based on a dynamic subtree partitioning strategy. This strategy allows the distribution of cached metadata hierarchically across a set of nodes adaptively. Each

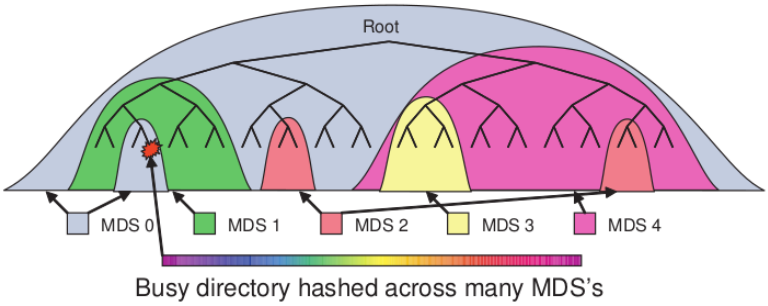


Figure 1.7: Subtrees mapping of the directory hierarchy to metadata servers [33]

MDS uses counters with an exponential time decay to measure the popularity of metadata within the directory hierarchy. When a process does an operation (like opening a file), the client sends a request to the MDS cluster, which traverses the file system hierarchy to translate the file name into the file inode, containing a unique inode number, owner, size, and other metadata. Any operation that affects an inode increment the counter associated with that and with all of its ancestors up to the root directory. In this way, a weighted tree representing the recent load distribution is obtained. In addition, all operations are journaled to a metadata journal and applied on a directory basis only when the metadata journal is stable.

Ceph dynamically maps the subtree directory hierarchy based on the resulting workload knowledge. The MDS load values are periodically compared, and the subtrees are migrated accordingly to keep an evenly distributed workload. Furthermore, Ceph uses the metadata popularity knowledge to avoid hot spots caused by many clients accessing the same directory by providing a wide distribution without losing the directory locality [33, p.6]. Clients learn the locations of metadata while traversing the file system. In this way, they can directly contact the appropriate MDS in the cluster [33, p.7].

Ceph Cluster Daemons

RADOS is the storage layer that underpins all the services in Ceph, like object, block, and file service [8]. Its purpose is to simplify the design and the implementation of the higher layers and to manage hardware details, data placements, migration, and rebalancing when the cluster grows or shrinks to provide a reliable and strongly

available storage layer [30]. Based upon RADOS, the Ceph storage cluster is a considerable collection of nodes that exchange information with each other to:

- write and read data,
- compress it,
- replicate it to ensure durability,
- redistribute it dynamically (backfilling),
- ensure data integrity,
- monitor and report on cluster health,
- recover from failures. [17]

Nodes in Ceph are frequently constituted by commodity hardware, and functionalities are provided through intelligent daemons: Ceph OSD daemon, Ceph Monitor, Ceph Manager.

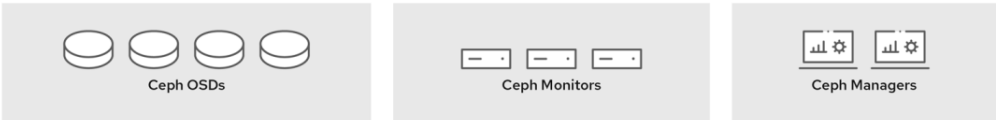


Figure 1.8: Ceph cluster daemons [17]

Ceph OSD Daemon (ceph-osd) stores data on behalf of Ceph clients. Each OSD usually manages a single local device, a traditional HDD (hard disk) or an SSD (solid state disk). Still, it can also be a combination of devices, including an HDD for most data and an SSD (or a partition) for some metadata [10]. OSDs manage the data they store with the default back-end BlueStore, the special-purpose storage back-end of Ceph built to overcome the limitations of the previous object store, File Store [8]. Each OSD utilizes the resource of a Ceph node, in terms of CPU, memory, and networking, to perform data replication, erasure coding, rebalancing, monitoring, and checking its state and other OSDs to report eventual failures to the monitor [17]. The number of OSDs depends on the amount of data to be stored, the size of each storage device, and the method adopted to assure data durability (replication or erasure coding) [8]. So there can be a small number of OSDs in a small cluster, like one or three, in a cluster proof-of-concept type deployment, or many thousands in a larger cluster with tens of petabytes of data [30]. OSDs are active storage daemons in the sense that they are not passive storage daemons. They cooperatively communicate with each other (peering) to ensure they are safely replicated and data is consistent, scrubbed, and rebalanced.

Ceph Monitor (ceph-mon) maintains a master copy of the Ceph cluster map with the current state of the cluster [17]. Monitor database requires a few gigabytes of storage in smaller clusters, while in larger ones, it requires tens to hundreds of gigabytes. Monitors use the Paxos algorithm to ensure agreement about the state of the cluster, thus meeting high consistency requirements. Managing the cluster state also includes managing cluster membership and authentication. Ceph provides its authentication system Cephx to authenticate users and daemons. Chephx enables the client and the monitor cluster to prove each other identities using a shared secret key. This authentication protocol operates like Kerberos, but unlike it, each monitor can authenticate users and distribute keys. In this way, there is no single point of failure when using Cephx. When a user invokes a Ceph client to contact a monitor, it returns to the client an authentication data structure similar to a ticket that contains a session key, which is encrypted with the user's secret key. The user is the only one able to initiate service requests from the Ceph monitors. The session key is then used to request the desired services from the monitor, providing a ticket that authenticates the client to the OSDs that handle the requested data. Tickets expire, like in Kerberos, so an attacker cannot leverage them. A user must be configured before using Cephx [17, p.13]. Monitors in a cluster, typically between three or seven, are distributed across different hosts or racks to obtain reliability, availability, and fault tolerance [30]. Literature refers to this as the "clustering of the Ceph Monitor." The number of monitors is usually odd to efficiently determine a majority during a Paxos algorithm [17, p.16].

Ceph Manager (ceph-mgr) acts as an endpoint and runs alongside monitor daemons. It handles the execution of read-only Ceph CLI queries, maintains detailed information about placement groups, processes metadata, and hosts them on behalf of the monitor to improve performance at scale. It also manages monitoring, orchestration, and plug-in modules [8][17]. There is typically one active manager in a cluster with some managers on standby, so if the active one fails or the host in which it is running falls, another monitor can take over (1 active, 1+ standby monitors per cluster) [30].

Objects, Pools, Placement Groups, and OSDs

The Ceph cluster receives data from Ceph clients through one of the three native interfaces or a custom interface implementation on top of librados. Using the RADOS protocol, data are stored as RADOS objects in logical partitions of the cluster called "Pools."

Administrators can create pools for particular data types or separate one group of users from another [17]. Pools usually map to some use case or deployment. Thus, it

is possible to have multiple specialized pools living within the same cluster, each with its storage policy. Furthermore, pools are crucial in determining how the Ceph cluster distributes and stores data because they handle complex operations transparent to the client, including data durability via replicas or erasure code chunks, data integrity, rebalancing, and recovery. Hence, they are not only logical partitions for storing object data [17]. Each pool has a "type" that describes the method adopted to assure the data durability, and it can be "replicated" or "erasure [8]." The former allows recovering from lost OSDs by keeping multiple copies of the objects, while the latter is like a generalized RAID5. The system administrator must choose wisely the pool type during its creation because it cannot be changed in the future.

Since handling many types of operations and managing data per-object basis on a cluster that might store millions of objects or more can present scalability and performance bottlenecks, Ceph shards a pool into a series of placement groups (PGs) to make object managing efficient. Generally, the number of PGs inside a pool is a fine-grained subset of the data [17, p.10-12]. Ceph stores replicas or erasure code chunks of an object within PGs to protect against data loss. The placement groups are distributed evenly and pseudo-randomly to the OSDs considering the map and the state of the cluster [17]. The CRUSH algorithm calculates where the objects should be stored, and it defines the Acting Set of the OSDs, which is a set of OSDs containing the PG replicas responsible for handling requests for the objects it has.

In summary, Ceph stores objects (or erasure code chunks of an object) in a deterministic way within PGs, which are replicated in a set of OSDs called Acting Set for the particular PG. The OSDs inside an Acting Set are referred to convection as the Primary, Secondary, and so forth. The Primary is the first OSD in the Acting Set. It is responsible for storing the first copy of the PG and is the only one that accepts client writes to the object of the contained placement group, for which it acts as the Primary. The OSD daemons that are part of an Acting Set may not always be in the up status. Thus, it is also necessary to consider the Up Set, a set of OSDs currently up in an Acting Set. The information provided by the Up Set is helpful for remapping PGs to other OSDs when an OSD fails. If the Primary of an Acting Set fails, the Secondary will become the Primary.

Generally, there are more placement groups than OSDs; thus, from a different perspective, a single storage device stores tens or hundreds of other placement groups from the same pool or other pools. Changes in the clusters cause changes in the cluster map (CRUSH map), which also changes the CRUSH computation for which OSDs store which PG. When this happens, the client can request from the Ceph Monitor an updated version of the cluster map to allineate its view of the cluster [17, p.14] and repeat the calculation.

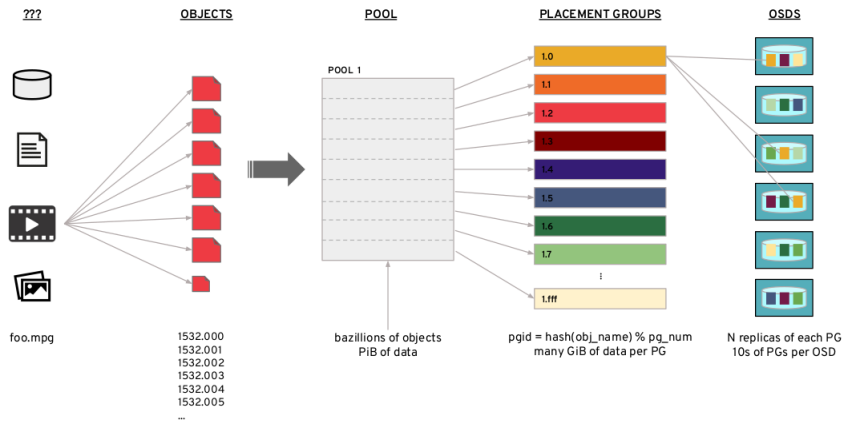


Figure 1.9: Objects - Pool - Placement Groups - OSDs mapping [30]

Ceph Input/Output Operations

When a Ceph client desire to retrieve or store data, it accesses the Ceph Monitor to get the latest copy of the cluster map (CRUSH map) and then uses the CRUSH algorithm to compute the object location (PG ID and Primary OSD). This way, Ceph does not need to maintain a centralized objects table to look up their location. That is called "calculated placement." The underlying idea consists of getting an initial map (CRUSH map) that concisely describes the cluster structure, how many and what servers are in the cluster, and the data distribution across them. Then, the location of the object is figured through a calculation that is a function of the state of the cluster and the name of the object [30]. So, the client's only inputs are the pool name, object ID (unique across the entire cluster), user name, and path to the secret key [17, p.7]. Behind that, the client can connect to the Primary OSD of the desired placement group in the pool, where it may perform read and write operations. For instance, consider the following pool and object-id: pool = "servicepool" and object-id = "service." After receiving the object ID, CRUSH hashes it and calculates the hash modulo of the number of PGs to get a PG ID (For instance, 32). Subsequently, the Primary OSD corresponding to the computed PG ID can be calculated. The client prepends the pool ID, obtained from the pool name (for instance, "servicepool" $\rightarrow$ 4), to the PG ID (getting the PG 4.32) and performs I/O operations by connecting directly to the Primary OSD in the Acting Set of the placement group bonded to the pool. Empowering the client and any party in the cluster to compute object locations makes them independent from a centralized per-object directory [33]. Furthermore, this approach is much faster and has no scalability limitation than querying the storage cluster for table look-up.

The OSDs receive data from clients and store them as objects in a flat namespace. Every object contains a globally unique identifier, an expandable amount of metadata,

and the data itself. The identifier is a unique 128-bit value associated with the object and is used to find it without knowing its physical location. Also, since the identifier is tied to its content, it allows for simple verification of potential bit-rot by recalculating the value and comparing it to the original. The metadata holds relevant information about the data, and there is no limit on their amount. Objects can contain both structured and unstructured data [17, p.7-8].

The Reason Behind Placement Group Architectural Choice

Examining three different design alternatives within a replication scenario, particularly in the event of failures and concurrent failures, helps to understand the reasons behind positioning placement groups between pools and storage devices instead of directly assigning objects to the storage device.

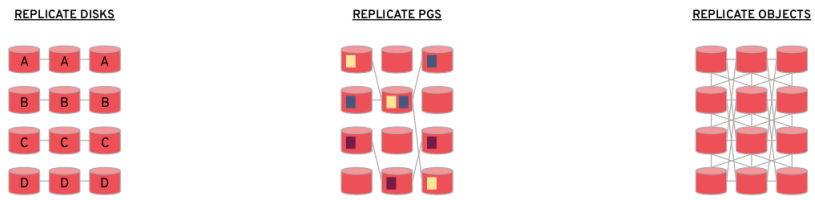


Figure 1.10: Three different design alternatives within a replication scenario [30]

The first and most straightforward approach is replicating disks in the system, doing three replications; thus, each device is mirrored. For this configuration to be achieved, the sizes of the devices must match. Otherwise, the smallest disk will define the maximum usable capacity for all three disks.

The second possible approach consists in replicating the placing group; thus, each PG is mirrored. In this scenario, it is possible to have different size devices because placement groups can be assigned randomly to devices, and the smaller devices do not define the maximum capacity. Still, they might have fewer placement groups than the larger devices.

Finally, it is possible to think of an extreme choice consisting of taking every single object in the system and randomly mapping it to different devices, ending up with a situation where every set of disks in the system shares replicas data with every other disk. Each object is mirrored, and a tightly, fully connected mesh of storage devices is obtained.

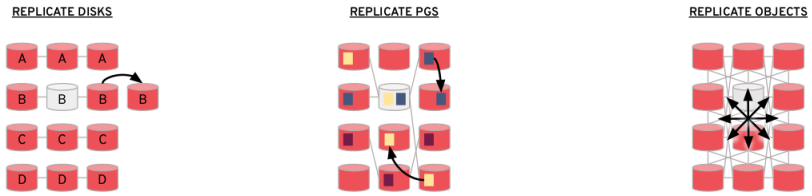


Figure 1.11: Scenario in which a device failure occurs [30]

Assuming a device fails:
In a disk replication scenario, a spare empty device is necessary to repair and compensate for the lost one. Moreover, the spare device must be of the appropriate size to make a new copy of the failed data onto the new disk. This scenario leads to some problems: having a set of spares is necessary, and spares must be the correct size. This results in the waste of resources in the period before failure. Furthermore, the recovery process is bottlenecked by the throughput of a single disk.

In the case of placement groups, when losing a device, copies of the lost placement groups remain on other still functioning devices. Also, it is possible to compensate the lost replicas of those placement groups to a new location chosen in an independent and pseudo-random way in the cluster. Recovery occurs faster because each PG replicates independently on different devices, making a parallel recovery possible.

In the object replication scenario, the failed device data is replicated with a little bit bits of it on every other device in the system. There is an all-all recovery process that has overhead implications.

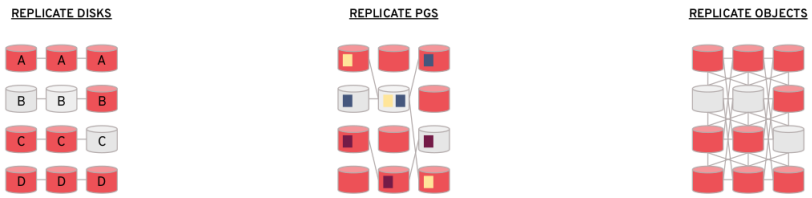


Figure 1.12: Concurrent failures scenario [30]

The situation becomes critical when considering the scenario where concurrent failures occur. In the disk replication scenario, it is likely that if three devices fail, they are not from the same replica set. Rarely all three failed replicas are of the same data, making it impossible to recover. In the object replication scenario, on the contrary, since objects are randomly placed on the cluster, almost every set of three devices has some data replicated just on those three devices, meaning there will always be some data loss. In the context of the placement group, some triple failures may not

result in data loss, while others may cause data loss.
Replication improves the reliability of the system at the expense of increased storage space required and bandwidth consumed. It is crucial to notice how the placement group strategy makes a parallel recovery reducing the recovery time compared to the disk replication scenario, where the throughput of a single disk after a device failure bottlenecks the recovery. These are the two extremes of the degree of parallelism that can be exploited during data recovery, respectively, in the declustered and clustered replica placement schemes. Declustered placement enables parallel recovery from all surviving devices, leading to low rebuild times. In contrast, clustered placement allows rebuilding only from a limited number of devices, which leads to higher rebuild times [31]. Using declustered replica placement, thus placement groups, represents a good compromise as it avoids the necessity to have a set of spare devices available, and it allows obtaining a balance between durability in the case of concurring failures and tolerable recovery time.

A complete strategy to keep data safe would be avoiding concurrent failures in the first place or ensuring that when they do happen, they do not lead to data loss. Ceph does this by creating a hierarchy of storage devices allowing system administrators to express the placement policy in terms of that hierarchy [30].

The CRUSH Algorithm, Map, and Ruleset

The CRUSH algorithm (Controlled Replication Under Scalable Hashing) "is a scalable pseudo-random data distribution function designed for a distributed object-based storage system that efficiently maps data objects to storage devices without relying on a central directory [35]." Given inputs including the cluster topology, pool parameters (e.g., replication factor), and the identifier of the placement group to be stored, this process produces an ordered list of OSDs responsible for storing the replicas of the placement groups. A hierarchical cluster map, describing the available storage resources in the cluster, controls the distribution of data objects across storage devices according to a per-device weight value. CRUSH generates a declustered distribution of replicas because the set of devices sharing replicas for one object is independent of all other objects. Furthermore, it utilizes a multi-input integer hash function, producing a mapping that is deterministic and independently calculable [35].

Ceph clients and OSDs use the CRUSH algorithm [17, p.9] to calculate object location. One of the properties of the CRUSH algorithm is that it generates a stable mapping, which means that the amount of data moved to rebalance the distribution after a topology change is proportional to the size of the change [17], maintaining the PGs in their position without forcing a reshuffle of all prior mapping, in contrast

as a traditional hashing approach [34]. This property arises from a design choice aimed at facilitating the addition or removal of storage while minimizing unnecessary data movement.

Changes in the cluster map or the state may automatically move PGs from one OSD to another [17, p.12]. Moving data around is expensive. In terms of performance, if a pool comprises too few PGs: each PG contains a large amount of data that needs to be moved simultaneously, leading to a performance bottleneck. At the same time, if a pool comprises too many PGs: cluster performance is affected by too much CPU and RAM usage in moving large quantities of tiny percentages of data [17]. In this scenario, performance depends on the number of PGs in a pool, which is set by the system administrators when creating or modifying a pool [17, p.10].

The cluster map consists of cluster membership, device state, and the mapping of data objects to devices. The mapping is initially defined regarding objects and placement groups and then further refined about PGs and devices [34]. Two fundamental components of the CRUSH map are devices and buckets, each characterized by unique numerical identifiers and weight values. The administrator assigns the weight, as a one-dimensional placement metric, to control the relative amount of data each device is responsible for storing. The weight of a device should be derived from its capabilities, while the weight of a bucket is the sum of the weights of the devices contained [35]. The CRUSH map is replicated both in clients and in very storage nodes. Each time the map changes, its epoch is incremented. These mechanisms facilitate the agreement between communicating parties regarding the current state of data distribution and enable them to assess whether their respective maps are up to date. Because cluster map changes may be frequent, updates are distributed as incremental maps, focussing on the difference between two successive epochs. The inter-OSD messages shift the map propagation to OSD, avoiding the burden of broadcasting map updates to all parties.

The device state includes information about the present network addresses of all the online and accessible OSDs (up) and indicates which devices are presently unavailable or offline (down). The OSDs usually are both up and in or both down and out. "In" and "out" are additional dimensions of OSD liveness; the former indicates OSDs included in the mapping and assigned to PGs, while the latter indicates OSDs that are not. Likewise, OSDs may also be down and in (currently unreachable) or up and out (online but idle) [34].
The versatile nature of buckets permits them to accommodate an arbitrary number of devices or additional buckets, allowing them to form interior nodes in a storage

hierarchy [35]. This hierarchical structure is defined from a directed acyclic graph and helps segregate the leaf nodes (OSDs) by their failure and performance domains. Storing copies of an object across different failure domains (for example, servers room, racks, and rows) enables the cluster to operate in a degraded state until the cluster recovers [17, p.10]. The hierarchy is arbitrary except for the leaf nodes, and a single CRUSH map can support multiple hierarchies with multiple root nodes [17].

Given the large variety of scenarios in which CRUSH might be used, placement rules are defined for each replication strategy or distribution policy employed [35]. These rules allow the storage system or administrator to specify how the object replicas are placed based on failure and performance domains for the pool. The CRUSH ruleset, in the same way as the pool type, is assigned by the administrator at the pool creation time.

The main factors for data placement are the number of PGs and the CRUSH ruleset. So when the client retrieves or stores data, Ceph identifies the CRUSH ruleset, a rule within the rule set, and the top-level bucket in the rule for storing and retrieving data [17, p.13]. It is crucial to notice that even though the client knows all the monitors, OSDs, and their current state by the last version of the CRUSH map, it does not know anything about the location of the objects.

The client identifies the Primary OSD containing the desired object on the Acting Set of a specific PG when Ceph processes the CRUSH rule. After that, the client can connect directly to the OSD and retrieve or store the object [17, p.13].

Replica Placement Pseudocode Algorithm [35]

Each rule in the CRUSH ruleset is a sequence of operations applied to the hierarchy that determines the number of replicas that need to be selected from the cluster and imposes certain constraints on the replica placement.

Considering x as an integer input value (typically an object identifier), CRUSH will output an ordered list $\vec{R}$ of n distinct storage targets. The following is a pseudocode of the CRUSH placement algorithm extracted from the "CRUSH: Controlled, Scalable, Decentralized Placement of Replicated Data" paper by Weil, Brandt, Miller, and Maltzahn [35].

```
                    CRUSH placement for object x
procedure TAKE(a)                          ▷ Put item a in working vector i⃗
        i⃗ ← [a]
end procedure

```

```
6  procedure SELECT(n,t)                                  ▷ Select n items of type t
7      o⃗ ← 0                                              ▷ Output, initially empty
8      for i ∈ i⃗ do                                        ▷ Loop over input i⃗
9          f ← 0                                          ▷ No failures yet
10         for r ← 1, n do                                ▷ Loop over n replicas
11             f_r ← 0                                    ▷ No failures on this replica
12             retry_descent ← false
13             repeat
14                 b ← bucket(i)                          ▷ Start descent at bucket i
15                 retry_bucket ← false
16                 repeat
17                     if "first n" then
18                         r′ ← r + f
19                     else
20                         r′ ← r + f_r n
21                     end if
22                     o ← b.c(r′, x)
23                     if type(o) ≠ t then
24                         b ← bucket(o)                  ▷ Continue descent
25                         retry_bucket ← true
26                     else if o ∈ o⃗ or failed(o) or overload(o, x) then
27                         f_r ← f_r + 1, f ← f + 1
28                         if o ∈ o⃗ and f_r < 3 then
29                             retry_bucket ← true        ▷ Retry local collision
30                         else
31                             retry_descent ← true       ▷ Otherwise retry descent from i
32                         end if
33                     end if
34                 until ¬retry_bucket
35             until ¬retry_descent
36             o⃗ ← [o⃗, o]                                  ▷ Add o to output o⃗
37         end for
38     end for
39     i⃗ ← o⃗
40 end procedure
41
42 procedure EMIT                                         ▷ Append working vector i⃗ to result
43     R⃗ ← [R⃗, i⃗]
44 end procedure
```

It is helpful to analyze the algorithm considering separately the three procedures that compose it:

- The **TAKE(a)** :
  This procedure selects a bucket within the storage hierarchy and assigns it to $\vec{i}$. The $\vec{i}$ will be utilized as input for the successive operations.

- The **SELECT(n,t)** :
  This procedure iterates over each element $i \in \vec{i}$, choosing n distinct items of type t in the current subtree analyzed. The type of storage devices is fixed, known, and used to discern between classes of buckets. More precisely, for each $i \in \vec{i}$ iteration, another iteration takes place over the $r \in 1, \ldots, n$ items while recursively descending the hierarchy through any intermediate buckets, pseudo-randomly selecting a nested item in each bucket through the function c(r,x) (different for each kind of bucket).

  The resulting n$|\vec{i}|$ distinct items are either re-inserted in $\vec{i}$ to form the input for the subsequent SELECT(n,t) or to the result $\vec{R}$ with an EMIT operation.

- The **EMIT**:
  This procedure appends the working vector $\vec{i}$ to the result vector $\vec{R}$.

Action	**Resulting $\vec{i}$**
take(root)	root
select(1,row)	row2
select(3,cabinet)	cab21 cab23 cab24
select(1,disk)	disk2107 disk2313 disk2437
emit	

Figure 1.13: A simple rule that distributes three replicas across three cabinets in the same row. [35]

For instance, considering a simple rule like the one in Figure 1.13, the algorithm begins at the root of the hierarchy and subsequently chooses in sequence: a single bucket of type "row" (row2), three distinct cabinets (cab21,cab23,cab24) nested in the previously selected bucket, and a single disk among the others for each of the previously selected cabinets. This approach allows replicas not only to be separated across container types but also to be constrained within them. Using rules comprising multiple TAKE and EMIT blocks enables the explicit selection of storage targets from distinct storage pools.

Focusing on the SELECT(n,t) operation: as previously stated, it traverses many levels of the storage hierarchy to locate n distinct items of the type t nested beneath the starting point. During this process, CRUSH may reject or reselect items using an alternate input r'. The reason for doing these actions relies on three possible causes:

- A collision occurred:
  An item was already selected in the current set (the SELECT(n,t) results must be distinct). The approach in this scenario involves utilizing a modified input r' during the initial stages of the recursive process. This modification is intended to facilitate a localized search and prevent any potential bias towards subtrees with a higher likelihood of collisions, thereby maintaining the overall integrity of the data distribution.

- A device is failed.

- A device is overloaded.

For failed and overloaded devices, items are uniformly redistributed across the storage cluster by restarting the recursion at the beginning of the SELECT(n,t). In the last two cases, the failed or overloaded devices are marked as such in the CRUSH map. Still, they are left in the hierarchy to avoid unnecessary data shifting because failures are typically temporary. In the case of negligent aging, the number of failed devices that were not replaced may become significant, leading to heavily degraded I/O performance.

Different requirements are present for parity and erasure coding schemes than replication. In replication, when a failure occurs, one of the replica targets, which already maintains a copy of the data, substitutes the old Primary for the specific acting set. On the contrary, the device placement in an erasure coding scheme is critical because each object chunk is stored in a different device. So, if a storage device fails, it should be replaced in place in the $\vec{R}$ output, such that other devices in the list retain the same position in $\vec{R}$. Lines 17-21 of the algorithm describes the stated scenario. In the case of a replication scheme, the new Primary can be found among the "first n" suitable targets by reselecting using r'=r+f (where f is the number of failed placement trials on the current SELECT(n,t)). While in the erasure coding scheme, the reselection takes place using $r' = r + f_r$ (where f_r is the number of failed attempts on r).

In contrast to conventional hashing techniques, in which changes in the number of cluster devices result in a massive reshuffling of contents, CRUSH defines four different kinds of buckets to represent non-leaf nodes in the cluster hierarchy.

Each of them is based on a different internal data structure and utilizes a different selection algorithm c(r,x) (line 22 in the algorithm) to address data movement resulting from the addition or removal of devices, pseudo-randomly choosing nested items during the replica placement process. There are four distinct types of buckets: uniform buckets, list buckets, tree buckets, and straw buckets. The uniform buckets are restricted to containing items of the same weight. On the contrary, the other types of buckets have no restrictions on the weight of items. They can contain a mix of items with any combination of weights.

- **Uniform Buckets**
  They are used to represent an identical set of devices. This type is useful because, in large systems, devices are often added in blocks of identical devices, leading them to simultaneously reach their end of life, so treating them as a unit is natural. In terms of performance, uniform buckets allow mapping replicas in constant time. Given x as the input value, r as the number of replicas, m as the bucket size, and p as a randomly chosen prime number with p>m, this is the selection algorithm c(r,x) used for uniform buckets:

  c(r,x) = (hash(x) + rp) mod m

- **List Buckets**
  They treat contents as a linked list. Replicas are added at the head of the list on top of the most recently added item, and compare their weights to the sum of the weights of all items in the list.

  c(r,x) = hash (x,r,item)
  The process continues recursively through the list until an item is chosen based on the current hash value. Not appropriate for large sets.

- **Tree Buckets**
  They treat content as a weighted binary search tree with items as the leaves. This approach reduces placement time against the linked list, efficiently managing larger devices or nested buckets. To select an item within a bucket starting at the root of the tree, CRUSH computes the hash of the input x, the replica number r, the bucket id, and the label at the current tree node. A comparison is made between the result obtained and the weight ratio of the left and right subtrees to determine the next node to be visited. This process is repeated until a leaf node is reached. By implementing this strategy, it becomes possible to ensure that the level of movement required remains within manageable limits while simultaneously enabling efficient mapping of data even when dealing with large buckets.

- **Straw Buckets**
  Through a process similar to a random drawing of straws, all items within the hierarchy are given an equal opportunity to compete for replica placement. This process ensures that each item is treated fairly and has an equal chance of being selected for replication. Also, it overcomes the limitation of the list and tree buckets, which either give certain items precedence or obviates considering the entire subtree of some items. Each straw length is scaled based on the weight of items by an $f(w_i)$ factor. In this way, heavily weighted items are more likely to win. Straw buckets result in optimal data movement.

  $c(r,x) = \text{maxi}(f(w_i)\ \text{hash}(x,r,i))$

These types of buckets represent a different tradeoff between calculation and reorganization efficiency. The expected cluster growth pattern guides the choice between them: if a cluster has a fixed bucket size, uniform buckets are faster; if a cluster is only expected to expand, list buckets are the best choice; if removal is expected and reorganization efficiency is critical straw buckets behaves better than tree buckets; three buckets is a well-balanced compromise that can be applied in a variety of situations.
CRUSH is designed to be fast in optimally distributing data to available resources.

OSDs are Intelligent Storage Devices

The cluster map, which encapsulates information about data distribution across the system, enables RADOS to delegate the responsibility for managing data redundancy, failure detection, and failure recovery to the OSDs that constitute the storage cluster. These mechanisms leverage the intelligence present in OSDs, by employing peer-to-peer-like protocols [34].

Replication and Erasure Coding

Given the massive scale of a petabyte or exabyte system, the likelihood of device failures at any given moment is relatively high, and a significant number of OSDs may be inoperable at any given time. Ceph can use data replication and erasure coding to maintain system availability and ensure data safety.

Replication

The replication strategy provides a level of fault tolerance, as data is stored on multiple OSDs. Specifically, the system utilizes placement groups as the unit of

data replication, with each PG being assigned to an ordered list of OSDs distributed to separate failure domains. The replication process is executed directly by the OSDs themselves [34], and it consists in making a deep copy of the placement group. In a typical write scenario, clients write the PG to the Primary OSD, finding the number of replicas it should store in the "osd_pool_default_size" setting and then replicating it [17]. This replication approach shifts replication traffic to the internal network of the cluster, simplifying client design. The client does not have to handle the complexity of the synchronization or serialization between PG replicas, which can become costly in the presence of other writers or failure and recovery operations [33].
RADOS implements three replication schemes: primary copy, chain, and a hybrid scheme called splay.

- The primary-copy scheme processes both reads and writes on the Primary OSD of the Acting Set and updates replicas in parallel.
- The chain replication scheme forwards the writes sequentially. It sends writes to the Primary OSDs (head) and reads to the tail, ensuring the reads reflect fully replicated updates.
- The splay replication scheme is a combination of the previous two analyzed strategies. It combines the parallel updates of primary-copy replication with the reads/writes role separation of the chain replication strategy.

Whichever scheme is chosen, the client sends I/O operations to a single OSD in the Acting Set. Then the cluster will ensure that the replicas are up to date and that serializability is preserved [34]. Considering a 3-replication strategy, after the client wrote the object on the Primary OSD, this triggers the write to the Secondary OSD and to a Tertiary OSD. The clients will receive an acknowledgment as a successful write operation from the Primary OSD only after it receives the acknowledgment from the Secondary and Tertiary OSDs [17].

It is worth emphasizing that RADOS allows modifying the replication factor at any point, providing flexibility and adaptability to changing needs or circumstances.

Erasure Coding

The erasure coding approach is a different reliability strategy; rather than storing deep copies of an object, an erasure-coded pool stores each object as K+M chunks. The Primary OSD in the Acting Set is responsible for sharding objects in chunks and maintaining the authoritative version of the placement group logs. An erasure code is a forward error correction (FEC) code that transforms a message comprising K

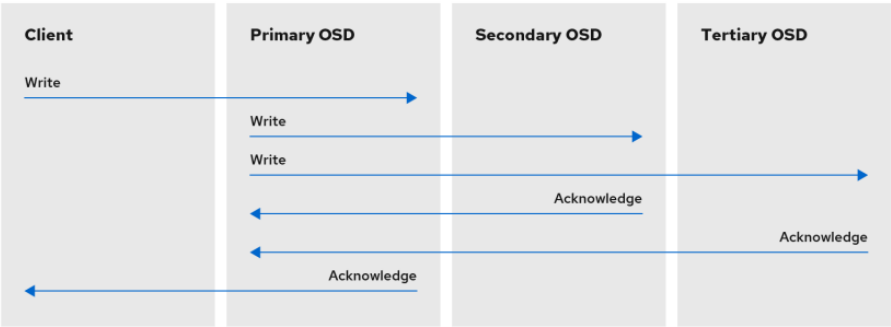


Figure 1.14: Replication: writing scenario [17]

data chunks into a "code word" consisting of N chunks and allows the recovery of the original object from a subset of N=K+M chunks. Ceph can load one of many erasure code algorithms [17, p.15]. The most commonly used is the Reed Solomon, which bases the recovery process on modular arithmetic and the Lagrange interpolation. K represents the number of chunks constituting the original amount of data (data chunks), while M represents the extra or redundant chunks (coding chunks) added to protect from failure. Since the number of total chunks (N) after the sharding process is also the number of OSDs needed to store all chunks, M represents the number of OSDs that can fail and still restore data. Thus, this strategy ensures resiliency to M=N-K arbitrary failures. For example, considering an erasure coding 4/6 with K=4 and N=6, Ceph will spread the N chunks across 6 OSDs, and even if 2 of them fail (M=2), it is possible to reconstruct the original object from the four remaining chunks. This strategy ensures a high level of fault tolerance. Furthermore, splitting data into chunks is independent of object placement, defined by the CRUSH ruleset and the erasure-code pool profile. Observing Figure 1.15, taken from the "RED HAT

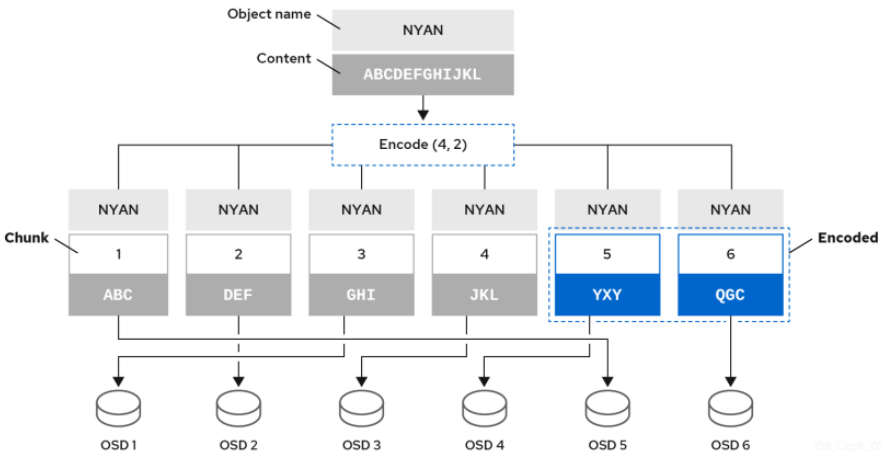


Figure 1.15: Erasure Coding (4,2) example for content "ABCDEFGHIJKL" [17]

CEPH STORAGE 5 Guide on Red Hat Ceph Storage Architecture," it is easy to notice how the "NYAN" object is sharded in 4 chunks. If the content length is not a multiple of K, the content will be padded. The coding chunks are YXY and QGC. The algorithm must preserve the order in which chunks are created in addition to the name of the object. In a recovery scenario, assuming that chunks 2 and 6 are missing, as soon as the algorithm has four chunks, it can rebuild the original content of the object. For instance, from chunk ABC, GHI, and JKL, with the help of YXY, it is possible to rebuild the "NYAN" object content (ABCDEFGHIJKL) [17].

Differences Between Replication and Erasure Coding Strategies

The advantage of the replication strategy is that in a recovery scenario, it is easy to create a new copy of an object from any existing replica. Simultaneously, the drawback of storing deep copies of objects is the associated cost of memory usage. In contrast, erasure coding significantly improves storage efficiency. Considering the example in the previous subsection, the initial four shards contain a complete copy of the original data, while the two additional shards provide redundancy with typically 50% storage overhead. This approach optimizes storage utilization, but it is less efficient in a recovery scenario because it is necessary to read all the surviving chunks and do some calculations to rebuild the original content. Thus, erasure coding is more expansive regarding network bandwidth and storage I/O than replication.

Furthermore, it is worth noting that erasure coding is especially advantageous when handling large objects that experience minimal changes. This advantage is because the infrequent data modification effectively masks the encoding overhead and the network bandwidth requirements. In such cases, erasure coding can provide efficient storage and reliable data recovery without significantly impacting system performance [30].

Failure Detection and Data Integrity

Timely failure detection is critical in ensuring data safety and integrity, but it can become increasingly challenging as a cluster scales to include thousands of devices. However, in the case of specific failures like disk errors or data corruption, OSDs can self-report these issues. This self-reporting mechanism enables OSDs to proactively communicate and notify the system about such failures, simplifying the detection process and enabling quick remedial actions. However, failures including the ones that make an OSD unreachable on the network require active monitoring [33, p.8]. When a failure occurs on the TCP socket, a limited number of reconnect attempts

are made before reporting a loss to the monitor cluster [34, p.4]. Ceph automatically performs many self-monitoring and management operations to resolve these, including Heartbeat, Peering, Rebalancing, and Recovery [17, p.18-19].

Heartbeat and Peering

As an OSD joins a cluster, it begins to report to Ceph Monitors on its status. Heartbeating consists of Ceph Monitors periodically pinging an OSD daemon to ensure it is running. Furthermore, this mechanism also empowers OSDs to know if a neighboring OSD is down and report this information to the Ceph Monitors to update the cluster map [17, p.18]. Intermittent lapses in OSD connectivity can cause transient or systemic problems. To filter this problem, Ceph considers two dimensions of OSD liveliness: whether the OSD is reachable (up-down status) or assigned data by CRUSH (in-out status). This combination of centralized and distributed detection avoids initiating data re-replication due to systemic problems by marking OSDs down but not out [33, p.8]. Additionally, it prevents any read operations from being processed by OSDs with an old map. If the OSDs servicing reads are not heard from other replicas in the cluster in H seconds, reads will be blocked [34, p.4].

Copies of a placement group have a status and are distributed across multiple OSDs of the Acting Set. These OSDs engage in peer-checking with one another to ensure a consensus on the status and the content of each PG replica to obtain a consistent view. The Primary of an Acting Set coordinates the peering process for a specific placement group. Each PG within an OSD for which it is not the current Primary causes a notification to be sent to its current Primary. This notification includes information about the considered PG, the most recent update, the PG log, and the epoch during which the PG successfully peered. Once the Primary is aware of the PGs for which it plays the Primary role, it generates a "Prior Set." This set includes the OSDs that have stored the PG since the last successful peering process. Using PG metadata for the entire Prior Set, the Primary determines the most recent update applied in each replica and starts the peering process to share missing log fragments with replica OSDs, to make them aware of which object the PG contained should include. Both peering and any subsequent recovery proceed independently for every PG in a pool [34, p.5].

Rebalancing and Recovery

Data rebalancing and failure recovery are driven entirely by cluster map updates and subsequent changes in mapping PG to OSDs. The cluster map changes as OSD failures happen and new storage is deployed. These changes, in turn, cause a shift in object placements. Since CRUSH places data evenly but pseudo-randomly, only

a small number of data is moved when a new OSD is added to the cluster. This amount usually equals the number of new OSDs divided by the total quantity of data in the cluster. During rebalancing in a Ceph cluster, the CRUSH algorithm ensures

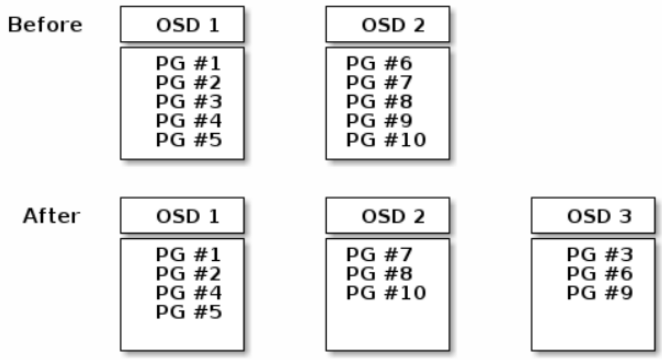


Figure 1.16: Rebalancing [8]

stability and minimizes disruptions. As the rebalancing occurs, many placement groups remain in their original configurations, and additional capacity is allocated to each OSD. This balanced data redistribution prevents load spikes on the newly added OSDs, resulting in a smooth and controlled rebalancing process [17, p.19].

OSDs maintain a version number and a log of recent changes for each PG. These pieces of information facilitate fast recovery. The recovery process consists of OSD daemons seeking the last copy of the data to bring its map up to date after a crash. For instance, supposing to have an Acting Set for the PG 1.23 constituted by "osd.1", "osd.2," and "osd.3", and that "osd.1", which is the Primary, is marked down. The "osd.2" takes over as Primary for the PG 1.23. When the "osd.1" recovers, it will request the latest map on boot and be marked as "up" by the monitor. As the "osd.2" receives the resulting map update, it will realize it is no longer the Primary for PG 1.23. Thus, "osd.2" sends the PG version number to the "osd.1", which will also retrieve the recent log of the PG 1.23 from "osd.2". Then "osd.1" tells "osd.2" that its contents are current and begins processing requests. While all this happens, "osd.1" retrieves any updated objects in the background. For objects within the placement group, the I/O operations are only allowed after the Primary OSD determines the correct state of the PG and shares it with the replica OSDs. It pushes any missing object to replica OSDs [34, p.5].

Since failure recovery is driven entirely by individual OSDs, when a placement group is affected by a failed OSD, the PG will be recovered in parallel with multiple replacement OSDs [33, p.9].

Scrubbing and CRC Checks [17, p.20]

Ceph provides two mechanisms to secure data against bad disk sectors and bit rot: Scrubbing and CRC Checks.

- **Scrubbing**
  Scrubbing is when Ceph OSDs Daemons compare object metadata contained within a placement group with its replicas in placement groups stored on other OSDs. It is performed daily and helps to catch bugs or storage errors. When performed weekly, it is called Deep Scrubbing, and it compares the whole object bit-for-bit, not limiting the comparison to the metadata fraction.

- **CRC Checks**
  When Ceph uses BlueStore as a storage implementation, data integrity can be provided by conducting a Cyclical Redundancy Check (CRC) on write operations. By storing the obtained CRC value, it can be used during read operations to be compared with the freshly generated CRC of the retrieved data. In case of a match, it ensures data integrity instantly.

Chapter 2

Troubleshooting and Data Loss

Having elucidated the fundamental aspects of Ceph, including its architecture and mechanisms, it is paramount to redirect the focus to common Ceph issues and how to troubleshoot them to get Ceph back up and running.
Issues in Ceph can span a broad spectrum ranging from the problems surrounding the monitoring process to troubles related to placement groups (PGs) and object storage daemons (OSDs). These problems require human intervention to be solved.

Troubleshooting monitors [11]

When a cluster encounters monitor-related problems, the foremost factor to investigate is whether or not the monitor daemon processes (ceph-mon) are running. In fact, it is typical for Ceph administrators to forget to start the monitor daemons or to restart them after an upgrade. Furthermore, checking that manager daemons (ceph-mgr) are also running is crucial.
Making sure those monitoring nodes are reachable can be the following initial troubleshooting action. If administrators cannot reach monitor nodes, this could be due, sometimes, to iptables rules blocking access to monitors or TCP ports. In these cases SSHing into the server and, if it succeeds, trying to connect to the ports of the monitor "tcp/3300" and "tcp/6789" could be a possible first step in understanding the reason for the unreachability issue.
To complete the initial troubleshooting actions, the administrator can verify whether or not monitors have formed a quorum. If the monitor answers a "status" request, it is guaranteed that a quorum has been reached. If the status request is unanswered or displays fault messages, likely, monitors are down or only a fraction of them are up, and that is not enough to form a quorum. In these cases, contacting each monitor individually may be helpful regardless of quorum formation.

Some monitor issues require further investigation after initial troubleshooting actions. The common ones are as follows:

- **Quorum is present, but at least one monitor is down**
  In scenarios where a monitor falls out of the quorum, its operational state can be one of the following: "probing," "electing," or "synchronizing." If the state is "probing," the monitor is still actively searching for other monitors. Usually, once started, the monitor maintains this state until it connects with the remaining monitors in the "monmap" (the monitor cluster map). Time spent in this state can vary. If the cluster has only one monitor (not recommended in production), the time interval spent in the probing state lapses instantaneously. While in the case of a multiple-monitor cluster, the time spent in this state will equal the one required to form a quorum. Finding the other monitors should not take long if a quorum has already been reached. So if one monitor is stuck in the probing state and the administrator has already performed the initial troubleshooting actions without success, it is likely that the monitor is trying to reach other monitors at the wrong address. Under these circumstances, it is necessary to check whether the monitor positions indicated in the "monmap" correspond to reality. If they do not, the monmap of the monitor is broken; if they do, the problem may be related to clock skew amongst the monitor nodes.

  If the state is "elected" instead, the monitor is in the middle of an election. In more recent releases of Ceph, the election process generally concludes rapidly. However, there are instances when monitors may become trapped in what is commonly called an "election storm." This phenomenon often indicates clock skew or inconsistencies among the monitor nodes.

  When a monitor enters the "synchronizing" state, it indicates that the monitor is aligning its state with the rest of the cluster to join the quorum successfully. The scale and current state of the cluster directly influence the duration required for synchronization. If the monitor state jumps back and forth from "synchronizing" to" electing," there is a problem.

  Finally, if the monitor state is "leader" or "peon," the monitor perceives itself to be within the quorum, while the remaining cluster, with certainty, acknowledges its absence from the quorum. This issue likely has a lot to do with clock skew.

- **Recovering broken "monmap" of a monitor**
  The safest path to recover a monmap is to obtain a copy of the monmap from the operational monitors and inject it into the affected monitor. By retrieving the intact monmap from the functioning monitors, it is possible to restore the necessary configuration and information to the corrupted or lost

monitor, enabling it to regain synchronization and functionality within the cluster. Monmap injection is also a practice used when monitor positions need to be updated. After updating the monmap, it is necessary to propagate it to the other monitors and inject it into each of them.

- **Clock skews**
  Clocks skew or inconsistencies among the monitor nodes can hardly affect monitor operations because the Paxos consensus algorithm requires strict time synchronization. To avoid this issue, the administrator can use clock synchronization tools like "Chrony" or the old "ntpd" on the monitor.

- **Monitor store failures**
  If a monitor fails due to the key/value store corruption, it is always possible to replace the corrupted monitor by replacing it with any survivor. If all monitors fail at once, although rare, the pieces of information stored in the OSDs can be used by the administrator to restore the monitor store.

Troubleshooting OSDs [12]

It is prudent to begin the OSDs troubleshooting by first assessing the condition of cluster monitors and network infrastructure. Network conditions can significantly influence the performance of OSDs. By prioritizing these preliminary checks and collecting OSDs data, a solid foundation for effective troubleshooting is established. Some common OSD issues that can arise in a Ceph cluster are as follows:

- **OSD not running**
  Restarting the Ceph OSD daemon resolves this kind of issue and allows the OSD to rejoin the cluster and recover. The causes of a non-running OSD can be different: non-compliant configuration file; data path or metadata path mismatch in configuration; reached the maximum number of threads on a node with many OSDs; Ceph third-party tools not compatible with the kernel; segment faults. If a drive failure or any other fault prevents the ceph-osd daemon from functioning or restarting, an error message will be logged in its respective log file. By examining the log file, it is possible to gain insights regarding the nature of the issue.

  By default, when OSD utilization approaches 85%, Ceph generates a health warning and refuses to participate in backfilling. Instead, when utilization reaches 95%, it will stop writing I/O to OSD. If the OSD fills up to 100%, it will likely crash and refuse to restart. In this situation, a possible solution is to

add capacity by adding more OSDs or deleting a few PGs directories in the full OSDs. Adding OSDs enables the cluster to redistribute data by moving data from overfull to less full OSDs [24]. On the other hand, deleting PGs directories will free up space on the overfull OSDs. However, it is crucial not to delete the same PG directory on another full OSDs because otherwise, this can lead to data loss.

- **OSD slow or unresponsive**
  This scenario is an OSD performance issue. The first thing to address is to check for network issues. Since Ceph relies upon networks for peering OSDs, replication, fault recovery, and heartbeat monitoring, networking issues can lead to OSDs latency and flapping OSDs. Other possible causes of the slow performance of an OSD can be due to:
  - **Drive Configuration**: Each drive should ideally host only one OSD to avoid performance bottlenecks. Multiple OSDs sharing a single drive can increase disk contention and degrade overall performance.
  - **Fragmented Disk or Bad Sectors**: Fragmentation of the underlying storage device of the OSDs or the presence of bad sectors can result in slower read and write operations.
  - **Monitors Co-residents with OSDs**: Co-locating monitors on the same host as OSDs can impact performance since they issue many *fsync()* calls.

    There may also be other causes: co-resident process; high logging levels; recovery throttling; insufficient ram; filesystem problems; and slow requests.

- **Flapping OSDs**
  When an OSD joins a cluster, its PGs initiate peering and monitor heartbeats. Separating public (front-end) and private (cluster/back-end) networks is recommended. When a private network experiences failure or degradation while the public network remains functional, OSDs use the public network to report each other down to the monitors while marking themselves up. After receiving these reports, the monitors send an updated cluster map with the affected OSDs marked down. Then, in turn, OSDs reply at this update to be up and not down, causing flapping OSDs, and the cycle repeats. A possible solution may be to carefully adjust the "mon_osd_down_out_subtree_limit," the "mon_osd_reporter_subtree_level," and the "mon_osd_min_down_reporters" settings.

Troubleshooting PGs [13]

Some of the common issues related to Ceph PGs include:

- **Placement groups never get clean**
  After creating a cluster, if its status remains "active," "active+remapped," or "active+degraded," and the "active+clean" status is never achieved, there is likely a configuration problem in the cluster. The starting point for evaluation is verifying whether there are fewer OSDs than replicas. This configuration issue can be due to the "default osd pool size" option being set to a value greater than the number of OSDs. It is recommended to run the Ceph cluster with more than one OSD and a pool size greater than one object replica.

- **Stack placement groups**
  When PGs get "stuck" for a long time in a non-optimal state, including "inactive," "unclean," and "stale," this may be due to a bigger problem. Specifically: for stuck "inactive" PGs, it is usually a peering problem; for stack "unclean" placement groups, it is a problem that prevents recovery from completing (including unfound objects); whereas for stuck PGs in a "stale" state, usually, the solution involves ensuring the correct OSD daemon is up and running.

- **Placement group down - peering failure**
  When there are complications during the peering process, preventing a PG from reaching an active and usable state, it is helpful to check the cluster health and interrogate the cluster to determine precisely why the PG is marked down.

- **Unfound objects**
  This issue happens when the cluster knows that some objects exist but has not found any copies. For instance, when a PG stores data on two OSDs (" osd.1" and "osd.2"), if the "osd.1" goes down, the "osd.2" will handle the writes until the "osd.1" comes back and starts recovering. Assuming that the "osd.2" goes down during recovery, before the recovery is complete, the "osd.1" knows that some new objects for the PG exist but cannot find them because their sole owner is down. Additional locations may not be included in the list where the objects could potentially exist. A possible solution is to query OSDs to find these objects. If all possible locations have been queried and objects are still lost, they must be marked as "lost."

- **Homeless placement groups**
  Homeless PGs are the ones for which the primary OSD in the Acting Set is down or fails to report to the monitor. The monitor that does not receives status updates from these PGs marks them as "stale." It is possible to identify

these PGs by checking the cluster health details. Restarting the OSD daemons in the last Acting Set of a "stale" PG will allow the cluster to recover that placement group [13][9].

- **Only a few OSDs receive data**
  This issue occurs when some placement groups are mapped to OSDs, while others are not. Creating pools with PGs whose amount is multiple of the number of OSDs can solve this problem.

- **Can not write data**
  If the minimum number of OSDs for a placement group is not running, Ceph will not allow writing data. This policy aims to preserve the remaining replicas of the PG because there is no guarantee that Ceph can replicate the data in such a situation.

- **PGs inconsistent**
  If an object within a PG becomes corrupted, Ceph will identify this issue during the scrubbing process and designate the PG that houses the corrupted object as inconsistent. The solution involves identifying the inconsistent PG by examining the health details of the cluster and listing the inconsistent objects associated with it. After that, to repair the inconsistent PG, Ceph overwrites the bad copy of the object with an authoritative one.

- **Erasure Coded PGs are not active+clean**
  This problem can arise when the cluster needs more OSDs, the CRUSH constraints cannot be satisfied, or CRUSH needs to give up before finding a mapping.

Data Loss

While most of the issues discussed in the preceding sections may affect the operations of the cluster without causing a complete outage or data loss, it is essential to acknowledge the scenarios that can lead to it. After an OSD fails, the likelihood of experiencing data loss becomes progressively higher until the data stored within it is completely recovered. The graph in Figure 2.1 shows the number of OSDs in a "down" state on the y-axis and the flow of time on the x-axis. It is possible to notice how, over time, the number of OSDs in a "down" state increases, forming spikes. The more OSDs are "down," the more OSDs are likely to go "down." This situation can be due to a lack of communication between peers, which, in turn, can be attributed to a software bug or cascading hardware failure. In the former situation, human intervention may be sufficient to solve the problem, restarting the

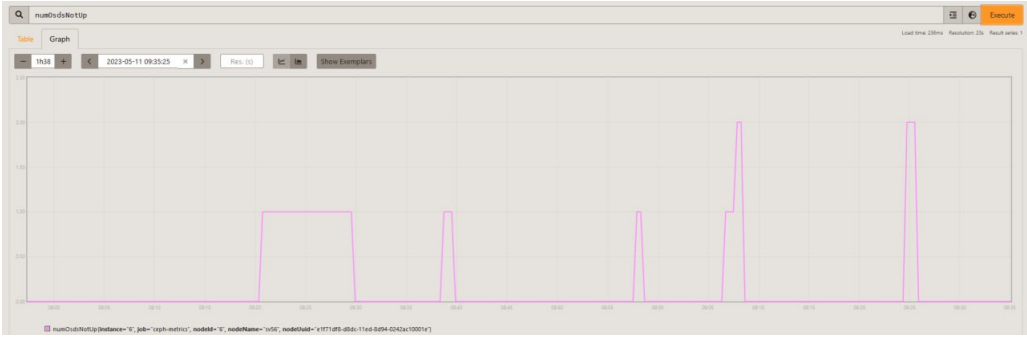


Figure 2.1: Prometheus graph of the number of OSDs not up (numOsdsNotUp)

affected OSDs in bulk to restore the exchange of packets between peers. In the latter, however, the recovery process is not trivial. As a result, the PGs contained within the affected OSDs are degraded. If this situation persists for a period that exceeds the "mon_osd_down_out_interval" (set to 600 seconds by default), the OSDs in a "down" state will be marked as "out." Subsequently, if in the cascading failure event, even the last replica of a placement group fails, the PG is marked as "unfound." As stated in the "troubleshooting PGs" section, in a case like that, not listed copies of an object may exist on the cluster. If not so, and all possible locations have been queried without finding them, the object must be marked as "lost."

For a more comprehensive understanding, exploring a scenario that results in permanent data loss within a single placement group is valuable. If an OSD experiences a failure:

- The replicas for each object within that placement group would immediately reduce from three to two, given that the placement group follows a 3-replication policy. The placement group status becomes "degraded."
- In response to this situation, Ceph initiates the recovery process for the affected placement group by selecting a new OSD to recreate the missing third copy of all the objects.
- In the meantime, another OSD containing the same placement group fails before the new OSD is fully populated with the third copy.
- As a result, certain objects within the PG will have only one surviving replica. Ceph picks yet another OSD to restore the desired number of copies.
- Furthermore, in an unfortunate turn of events, the third OSD, responsible for storing the same placement group, experiences a failure before the recovery process can be completed. As the only remaining copy of the placement group, the PG and all its objects are permanently lost [8, p.292].

Thus, in a Ceph cluster, the main reason for outages and data loss is directly connected with the loss of several OSDs that quickly exceed the replication level because of a software or hardware failure. These objects are lost if these OSDs do not come back online, and Ceph cannot recover them in-between failures of OSDs. The likelihood of this happening increases significantly when the replication algorithm crashes, hindering recovery. This failure scenario can be attributed to the difficulty in finding the appropriate OSDs to accommodate the replicas and, at the same time, ensuring compliance with the policy that enforces placement group collocation in different failure domains.

The time required for a complete recovery depends entirely on the architecture of the Ceph cluster. If the size of the cluster is small, the number of placement groups has little influence on the durability of the data. Moreover, the duration of the recovery process will not be affected in any way. However, if the cluster gets more extensive, as the number of OSDs increases, each will contain fewer PGs. Since having fewer PGs per OSD means that each will participate in fewer PGs during recovery, the recovery time is reduced, and data durability is improved. Nonetheless, it is essential to mention that the recovery time increase when the quantity of OSDs significantly surpasses that of the placement groups. The reason is that the quantity of OSDs involved equals the number of PGs contained within a OSD multiplied by the replication factor, which will be smaller than the real OSDs quantity.

In light of the preceding, no matter how short the recovery time is or how many OSDs are in the cluster, there is always a chance for a second OSD to fail while the recovery process is in progress. Data durability and distribution among all OSDs call for more placement groups, but their number should be reduced to the minimum to save CPU and memory. In fact, Ceph is self-contained in failure recovery when resources are available. Since it is deployed on commodity hardware, the risk of data loss in Ceph, although relatively low, still exists.

If an OSD fails due to a disk failure, recovery is unlikely unless expensive disk recovery services are utilized. Even then, there is no guarantee that the recovered data will be consistent. On the contrary, if an OSD fails due to a software bug, the outcome is probably more optimistic. Still, the process is complex and time-consuming because, most of the time, the software bug results from uncaught exceptions leaving the OSD in a state where it cannot be recovered. Looking at the network level, even though Ceph network architecture is spine-leaf, which allows for redundant paths between nodes, the progressive failure of routers on the switching layers is not unlikely. These network failures can lead to the perception of losing

a set of OSDs. While Ceph is designed to be largely self-sufficient in managing its operations and recovering from various failure scenarios, the data loss problem remains complicated. It depends on many factors, including:

- the number of OSDs in the cluster;
- the OSDs distribution pattern across buckets into the CRUSH map;
- the various types of replication and fault domain policies utilized for different objects within the same pool;
- the utilization rate of storage devices;
- and the location of the OSDs in the cluster network.

In this complex, heterogeneous, and ever-changing scenario, it is challenging for a Ceph system administrator to individuate the best Ceph deployment to mitigate data loss a priori. The reason behind this is that the deployment reliability depends on both the hardware and the mapping of the software system onto it. Furthermore, it is difficult to know how sensitive the data is to the network or bucket failures.

There is a clear requirement for tools capable of querying the cluster to determine how the system will evolve in case of failures and enable the Ceph administrator to effectively assess the evolving data loss probability in the face of cascading failures. Achieving this goal necessitates mapping the current Ceph cluster to find which PGs, OSDs, and buckets can be affected by a cascading failure in the network infrastructure or the Ceph hierarchy. It is also necessary to consider how different replication policies come into play and how their application dynamically changes as the cluster evolves.

As stated in the "Ceph" chapter, each OSD is backed by a single storage device. Such storage device can be a traditional hard disk (HDD) or solid-state disk (SSD). Each device has a lifespan for which the manufacturer guarantees optimal functionality. This lifespan is calculated differently for HDDs and SSDs.
HDDs feature rotating disks that are subject to progressive wear and tear, leading to a reduction in their operational lifespan. Consequently, an HDD lifespan is typically assessed by considering its cumulative hours of use.
On the other hand, SSDs rely on integrated circuits for data storage and experience decreased reliability over time as the transistors wear out and their charge-holding capacity diminishes. Thus, their lifespan depends mainly on the TeraBytes Written over time (TBW) [32] [5]. By evaluating the current lifespan of storage devices within the Ceph cluster and analyzing their degradation rate, accurate predictions can be made regarding the future degradation of specific devices, particularly OSDs.

A tool can be developed to leverage this information to enable Ceph administrators to anticipate the progression of data loss probability. Consequently, administrators can gain valuable insight that empowers them to proactively take necessary actions and operate efficiently until potential failures occur.

Service-Related Data Loss Assessment

Companies utilize Ceph to fulfill their storage requirements and facilitate the delivery of various services. The object storage system offered by Ceph can be leveraged for deploying web applications that need to store and retrieve large amounts of unstructured data or involving media streaming. In addition, block storage and file storage enable the leverage of Ceph for other storage needs, including persistent block storage for virtual machines or shared file storage for application data. These services are bonded to the pool concept in Ceph.

Ceph currently needs to provide the administrator with a clear view of the pools affected by the data loss. By obtaining a comprehensive understanding of the data loss of the pools, administrators could determine the most suitable Ceph deployment that provides data protection against failures and aligns with the requirements of users or applications reliant upon it. Therefore, it is essential to have a time-dependent and service-oriented view of the Ceph cluster during faults or in their anticipation.

The next chapter analyzes the incremental data loss assessment algorithm and the data loss forecasting algorithm in detail. The former helps the administrator to understand which PGs in a pool are affected by bucket or router failures in the cluster. At the same time, the latter allows the administrator to predict which PGs will be involved in future OSD failures, considering the current cluster deployment and the associated storage device lifetime. These algorithms are integrated into a Server component deployed within a Ceph cluster, and their results can be requested through REST API or RPC queries. By leveraging current information obtained from the hosting machine in the cluster, this server component offers valuable insights into the current or future data loss probability for the pools in the cluster.

Chapter 3

Service-Related Data Loss Evaluation System

Overview

The underlying idea is to create a versatile Server component that supports multiple inputs and outputs, which the administrator can easily deploy within a Ceph cluster. The primary role of this component is to calculate the data loss probabilities of the pools and provide predictions based on the current state of the deployed cluster. The Server component retrieves the current cluster state with each request through the available Ceph commands. In input, it accepts REST API requests or RPCs; in output, it produces its results as JSON responses or Prometheus metrics. Administrators can sporadically query the Server component through a Proactive Client to obtain data loss probabilities of the pools or their predictions during maintenance operations. At the same time, using a dedicated Reactive Client, it is possible to periodically query the server component to provide real-time information to a monitoring tool. The collective integration of these components forms a comprehensive Service-Related Data Loss Evaluation System.

Chosen Programming Languages and Tools

Golang

Golang is a statically typed, compiled, high-level programming language [20]. It is geared toward creating servers and back-end systems and focuses on concurrency and efficiency. The simplicity and the powerful performance that distinguish it make Golang the ideal solution to implement the Server component deployed on the Ceph cluster.
Also, with its clean syntax and the inclusion of the standard library package "net/rpc,"

[21] it is an optimal solution to implement the Proactive Client, which uses RPCs (Remote Procedure Calls) to query the server on data loss probability of pools and their predictions.

Python

Python is a dynamically typed, high-level, general-purpose programming language [22]. It is selected as the programming language for implementing the Reactive Client to ensure future adaptability within a periodic Apache Airflow monitoring workflow. Apache Airflow is a powerful tool that enables the description, execution, and monitoring of workflows using the Directed Acyclic Graph (DAG) concept. This concept allows the representation of tasks and their dependencies as Python scripts [1].

Prometheus [27][26]

Prometheus is an open-source monitoring and alerting system. It is designed to collect and record metrics as time series, which means recording the changes in numeric measurements over time.
Prometheus operates based on a pull model, periodically scraping metrics data from instrumented jobs. These can be scraped directly through configured targets, including application endpoints, or via an intermediary push gateway for short-lived jobs. The scraped samples are stored locally within the Prometheus server node.

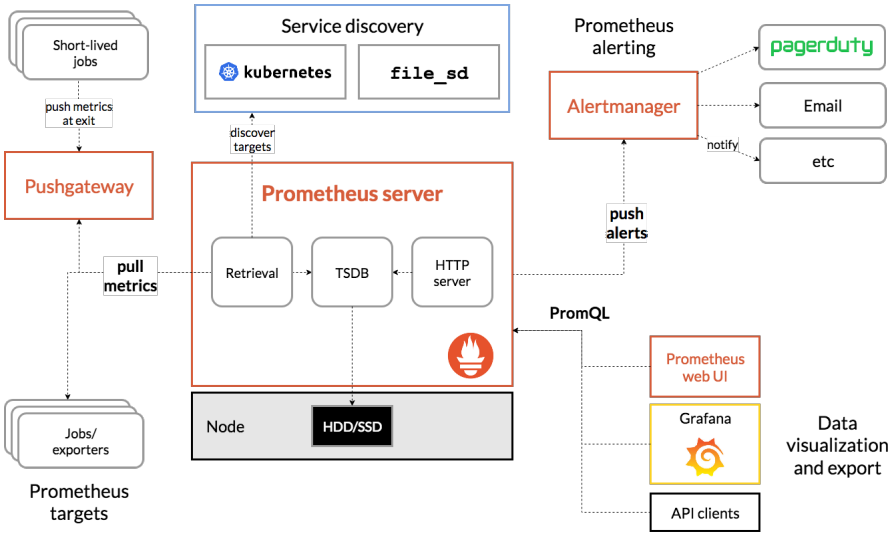


Figure 3.1: Prometheus architecture [27]

The architecture and ecosystem of Prometheus consist of multiple components.

The main ones are the Prometheus Server node that scrapes and store time series data and the client libraries designed to integrate and instrument application code. Other components are: push gateway; special-purpose exporters; alert manager; and other support tools. Most of these components are written in Golang and are optional.

A standout feature of Prometheus is its powerful query language called PromQL. PromQL can extract specific metrics that can be visualized in customized monitoring dashboards using Grafana or other API clients.
Metrics play a crucial role in monitoring an application or server. Prometheus offers four core metric types:

- **Counter**
  A counter is a metric that keeps track of a cumulative value, which can only go up or be reset to zero upon restart. It cannot be used to expose a value that can decrease.

- **Gauge**
  A gauge is a metric that keeps track of a single numerical value that can arbitrarily go up and down.

- **Histogram**
  A histogram captures and counts sampled observations and can also provide a sum of all observed values.

- **Summary**
  It is similar to a histogram metric, which calculates quantiles over a sliding time window. A quantile is a value that leaves a certain percentage of the given values above or below a certain threshold.

Grafana [36]

Grafana is a powerful and flexible tool used for visualizing and monitoring data. It allows the creation of interactive dashboards that display metrics and logs from different sources in an attractive and customizable way. Grafana supports many data sources, including monitoring systems like Prometheus.

Deploying a Ceph Cluster for Testing Purposes

During the development process, it was necessary to use a Ceph installation on a small cluster made up of virtual machines to test the Service-Related Data Loss Evaluation System. Ceph 17.2.5 Quincy (stable) was installed using the Cephadm

tool. This deployment tool creates a new cluster by "bootstrapping" on a single host, expanding the cluster to include additional hosts, and deploying the needed services. It uses Docker to run all Ceph daemons, and the deployment management is done via an SSH (Secure Shell) connection.

Four Kernel-based Virtual Machines (KVM) were generated to establish a basic configuration for the cluster nodes. These include a node responsible for hosting the initial "monitor daemon" where the "cephadm bootstrap" command was executed (named cephadm), as well as three nodes representing the OSDs (ceph0, ceph1, ceph2). Each node runs Fedora Server 37 on a KVM hypervisor with a size of 20GB. The OSDs are additionally associated with 10GB of HDD storage. For this installation, the allocation of storage capacity to individual hosts considered the existing storage capabilities of the utilized laptop.

The file "/etc/hosts" of each node was modified to include the address-host mapping of the remaining cluster nodes. Furthermore, a reachability check was performed from each node to every other node. To reduce performance spikes associated with the TCP protocol, the "net.ipv4.tcp_timestamps" line was set equal to zero in the "/etc/sysctl.d/99-ceph.conf" file.

Following that, labels were assigned to each host added to the cluster, indicating to the orchestrator which daemons should be deployed on each of them. Subsequently, the orchestrator deployed these daemons through the "ceph orch apply" command.

```
[ceph: root@cephadm /]# ceph orch host ls
HOST     ADDR             LABELS          STATUS
ceph0    192.168.122.80   mgr
ceph1    192.168.122.81   mon
ceph2    192.168.122.82   mgr mon
cephadm  192.168.122.224  _admin mon mgr
4 hosts in cluster
```

Figure 3.2: Output of "ceph orch host ls" command

This deployment was chosen to ensure the quorum across the monitors and establish redundancy among managers.
Additionally, it was possible to test the Service-Related Data Loss Evaluation System in a production environment provided by Aucta Cognitio, a reputable company in the Cloud, Big Data, and IoT domains based in Catania, Sicily.

Parsing Functions and Mapping Cluster Hierarchy

It is crucial to map the existing Ceph cluster to determine the potential impact on PGs, OSDs, and buckets, of a cascading failure in the network infrastructure or Ceph hierarchy. As stated in the "Troubleshooting and Data Loss" chapter, this mapping

process allows for a comprehensive understanding of the specific buckets that may be affected. The information required to perform the mapping can be obtained through the statistics provided by Ceph using the following commands:

```
#ceph pg dump --format=json
```

```
#ceph osd dump --format=json
```

```
#ceph osd tree --format=json
```

After creating the structures that mirror the data of JSON files, the unmarshalling into these was performed through the following functions:

```go
func ReadPgDumpJson(jsonFileName string) s.PgDumpOutputStruct {
   jsonFile, err := os.Open(jsonFileName)

   if err != nil {
      fmt.Println(err)
   }

   defer jsonFile.Close()

   // read the opened jsonFile as a byte array.
   byteValue, _ := io.ReadAll(jsonFile)

   var pgStruc s.PgDumpOutputStruct

   json.Unmarshal(byteValue, &pgStruc)

   return pgStruc
}
```

```go
func ReadOsdTreeJson(jsonFileName string) s.OsdTreeOutputStruct {
   jsonFile, err := os.Open(jsonFileName)

   if err != nil {
      fmt.Println(err)
   }

   defer jsonFile.Close()
```

```go

    // read the opened jsonFile as a byte array.
    byteValue, _ := io.ReadAll(jsonFile)

    var osdStruc s.OsdTreeOutputStruct

    json.Unmarshal(byteValue, &osdStruc)

    return osdStruc
}
```

```go
func ReadOsdDumpJson(jsonFileName string) s.OsdDumpOutputStruct {
    jsonFile, err := os.Open(jsonFileName)

    if err != nil {
        fmt.Println(err)
    }

    defer jsonFile.Close()

    // read the opened jsonFile as a byte array.
    byteValue, _ := io.ReadAll(jsonFile)

    var osdStruc s.OsdDumpOutputStruct

    json.Unmarshal(byteValue, &osdStruc)

    return osdStruc
}
```

The temporal analysis of the information contained in these files enables a comprehensive view of the examined cluster. Through the implementation of the parsing functions, it was possible to acquire statistics about the following:

- Pools, PGs, and OSDs;
- individual node structs;
- and the distribution of the OSDs in the various buckets.

Herein lies a selection of the principal functions aforementioned, while the remaining ones are available on GitHub.

```go
func GetPgs(pgDumpOutput s.PgDumpOutputStruct) []string {

   pgStats := pgDumpOutput.PgMap.PgStats
   pgSlice := []string{}

   for _, s := range pgStats {
      pgSlice = append(pgSlice, s.Pgid)
   }

   return pgSlice
}
```

```go
func GetPools(pgDumpOutput s.PgDumpOutputStruct) []string {

   pgSlice := GetPgs(pgDumpOutput)
   poolSlice := []string{}

   for _, s := range pgSlice {
      poolId := strings.Split(s, ".")[0]

      poolSlice = append(poolSlice, poolId)
   }

   poolSlice = utility.RemoveDuplicateStr(poolSlice)
   return poolSlice
}
```

```go
func GetOsds(pgDumpOutput s.PgDumpOutputStruct) []int {

   osdPgMap := GetOsdPgMap(pgDumpOutput)
   osds := []int{}

   for osd := range osdPgMap {
      osds = append(osds, osd)
   }

   return osds
}
```

By specifying the name of the desired node, the following function retrieves the node struct containing its information:

```
func GetNode(nodeName string, osdTreeOutput s.OsdTreeOutputStruct)
    (nodeStruct s.NodeStruct) {
   nodeOsdTreeStats := osdTreeOutput.Nodes

   for _, node := range nodeOsdTreeStats {
      if node.Name == nodeName {
         nodeStruct = node
      }
   }
   return
}
```

```
type NodeStruct struct {
   ID          int    `json:"id"`
   Name        string `json:"name"`
   Type        string `json:"type"`
   TypeID      int    `json:"type_id"`
   Children    []int `json:"children,omitempty"`
   PoolWeights struct {
   } `json:"pool_weights,omitempty"`
   DeviceClass    string `json:"device_class,omitempty"`
   CrushWeight    float64 `json:"crush_weight,omitempty"`
   Depth          int     `json:"depth,omitempty"`
   Exists         int     `json:"exists,omitempty"`
   Status         string `json:"status,omitempty"`
   Reweight       int     `json:"reweight,omitempty"`
   PrimaryAffinity int   `json:"primary_affinity,omitempty"`
}
```

The GetDistributionMap function is the core of the RiskCalculator and IncrementalRiskCalculator functions, analyzed in the subsequent sections. This function outputs a map where each key represents a specific bucket, and the corresponding value indicate the distribution of that bucket throughout the whole hierarchy (from the root to OSDs). The function produces a distinct map for every bucket type within the hierarchy when provided with the corresponding bucket type as choosenMapKey input. For ease of discussion, a section of the GetDistributionMap function is shown in the next page. The remaining portion of the code can be accessed and found on GitHub.

```go
func GetDistributionMap(choosenMapKey string, pgDumpOutput
    s.PgDumpOutputStruct, osdTreeOutput s.OsdTreeOutputStruct)
    map[string]s.BucketDistribution {
   osdTreeStats := osdTreeOutput.Nodes

   bucketSlice := []string{"root", "region", "zone", "datacenter", "rack",
       "chassis", "host"}

   mappingSlice := []map[string][]string{}
   distributionMap := make(map[string]s.BucketDistribution)

   for _, bucket := range bucketSlice {
      var bucketName string
      var bucketChildren []int
      tempMap := make(map[string][]string)

      for _, node := range osdTreeStats {
         if node.Type == bucket {
            bucketName = node.Name
            bucketChildren = append(bucketChildren, node.Children...)
         }

         for _, child := range bucketChildren {
            if node.ID == child {
               tempMap[bucketName] = append(tempMap[bucketName], node.Name)
            }
         }
      }

      mappingSlice = append(mappingSlice, tempMap)
   }

   rootMap := mappingSlice[0]
   regionMap := mappingSlice[1]
   zoneMap := mappingSlice[2]
   datacenterMap := mappingSlice[3]
   rackMap := mappingSlice[4]
   chassisMap := mappingSlice[5]
   hostMap := mappingSlice[6]

```

```
38    switch choosenMapKey {
39    case "host":
40       for host := range hostMap {
41
42          hostName := host
43          chassisName := GetBucketName(hostName, chassisMap)
44          rackName := GetBucketName(chassisName, rackMap)
45          datacenterName := GetBucketName(rackName, datacenterMap)
46          zoneName := GetBucketName(datacenterName, zoneMap)
47          regionName := GetBucketName(zoneName, regionMap)
48          rootName := GetBucketName(regionName, rootMap)
49
50          if _, isPresent := distributionMap[hostName]; !isPresent {
51             osdSlice := hostMap[hostName]
52             hostDistribution := s.BucketDistribution{Root: rootName,
                  Region: regionName,
53                Zone: zoneName, Datacenter: datacenterName, Rack: rackName,
54                Chassis: chassisName, Host: hostName, Osd: osdSlice}
55
56             distributionMap[hostName] = hostDistribution
57          }
58       }
59 // ... Code matching the remaining types of buckets in the hierarchy ...
60    }
61    return distributionMap
62 }
```

For instance, for a given input "host" as choosenMapKey, an element of the resulting map output will be the following:

```
1 # --- An element of the GetDistributionMap Output ---
2 sv61:{default 3640c696-default-region 3640c696-default-zone
    ac4a9bba-sjc-hq b076020c-default-rack 9ddc43d4-c8250fd30n10355 sv61
    [osd.1 osd.2 osd.4 osd.6]}
```

Furthermore, functions were developed to map Pools with PGs and OSDs.

Mapping PG-OSD:

```
func GetPgOsdMap(pgDumpOutput s.PgDumpOutputStruct) map[string][]int {

   pgStats := pgDumpOutput.PgMap.PgStats
   pgIdOsdMap := make(map[string][]int)

   for _, s := range pgStats {
      pgOsdSlice := s.Up
      pgIdOsdMap[s.Pgid] = pgOsdSlice
   }

   return pgIdOsdMap
}
```

Mapping OSD-PG:

```
func GetOsdPgMap(pgDumpOutput s.PgDumpOutputStruct) map[int][]string {

   pgIdOsdMap := GetPgOsdMap(pgDumpOutput)
   osdPgMap := make(map[int][]string)

   for key, osdSlice := range pgIdOsdMap {
      for _, osd := range osdSlice {
         if _, ok := osdPgMap[osd]; !ok {
            osdPgMap[osd] = []string{}
         }

         osdPgMap[osd] = append(osdPgMap[osd], key)
      }
   }

   return osdPgMap
}
```

The last was easily obtained by exploiting the GetPgOsdMap function. Also, to obtain a service-related mapping, the mapping OSD-Pool-PGs was implemented:

```
type OsdPoolTuple struct {
   Osd, Pool interface{}
}
```

```go
// {(osd,pool):[pgs],...}
func GetOsdPoolPgMap(pgDumpOutput s.PgDumpOutputStruct)
    map[OsdPoolTuple][]string {

   osdPgMap := GetOsdPgMap(pgDumpOutput)
   osdPoolPgMap := make(map[OsdPoolTuple][]string)

   for osd := range osdPgMap {
      for _, pg := range osdPgMap[osd] {
         poolId := strings.Split(pg, ".")[0]
         osdPoolTuple := OsdPoolTuple{osd, poolId}

         if _, ok := osdPoolPgMap[osdPoolTuple]; !ok {
            osdPoolPgMap[osdPoolTuple] = []string{}
         }

         osdPoolPgMap[osdPoolTuple] = append(osdPoolPgMap[osdPoolTuple], pg)
      }
   }

   return osdPoolPgMap
}
```

Moreover, additional functions were developed to calculate which PGs have the higher probability of losing data due to the fault of a single OSD or a slice of them:

```go
func GetAffectedPgs(faultOsd int, pgDumpOutput s.PgDumpOutputStruct)
    []string {
   osdPgMap := GetOsdPgMap(pgDumpOutput)

   affectedPgSlice := []string{}

   affectedPgSlice = append(affectedPgSlice, osdPgMap[faultOsd]...)

   return affectedPgSlice
}
```

```go
func GetAffectedPools(faultOsd int, pgDumpOutput s.PgDumpOutputStruct)
    []string {
```

```go

    osdPgMap := GetOsdPgMap(pgDumpOutput)

    affectedPoolSlice := []string{}

    for _, pg := range osdPgMap[faultOsd] {
        poolId := strings.Split(pg, ".")[0]

        affectedPoolSlice = append(affectedPoolSlice, poolId)
    }

    return utility.RemoveDuplicateStr(affectedPoolSlice)
}
```

```go
func GetTotalAffectedPgsAndPools(faultOsdSlice []int, pgDumpOutput
    s.PgDumpOutputStruct) ([]string, []string) {

    totalAffectedPgs := []string{}
    totalAffectedPools := []string{}

    for _, osd := range faultOsdSlice {
        affectedPgs := GetAffectedPgs(osd, pgDumpOutput)
        affectedPools := GetAffectedPools(osd, pgDumpOutput)

        totalAffectedPgs = append(totalAffectedPgs, affectedPgs...)
        totalAffectedPools = append(totalAffectedPools, affectedPools...)
    }

    totalAffectedPools = utility.RemoveDuplicateStr(totalAffectedPools)

    return totalAffectedPgs, totalAffectedPools
}
```

```go
func GetPgNumberOfAffectedReplicaMap(faultOsdSlice []int, pgDumpOutput
    s.PgDumpOutputStruct) map[string]int {

    totalAffectedPgs, _ := GetTotalAffectedPgsAndPools(faultOsdSlice,
        pgDumpOutput)
    pgNumberOfAffectedReplicaMap := make(map[string]int)

    for _, pg := range totalAffectedPgs {
```

```
7        if _, ok := pgNumberOfAffectedReplicaMap[pg]; !ok {
8           pgNumberOfAffectedReplicaMap[pg] = 1
9        } else {
10          pgNumberOfAffectedReplicaMap[pg] += 1
11       }
12    }
13
14    return pgNumberOfAffectedReplicaMap
15 }
```

```
1  func GetPgsWithHighProbabilityOfLosingData(pgNumberOfAffectedReplicaMap
       map[string]int, pgDumpOutput s.PgDumpOutputStruct, osdDumpOutput
       s.OsdDumpOutputStruct) (inHealthPgs []string, goodPgs []string,
       warningPgs []string, lostPgs []string) {
2
3     for pg, numLostReplicas := range pgNumberOfAffectedReplicaMap {
4
5        pgTotalReplicas, _, _ := GetOsdNumberForPg(pg, osdDumpOutput)
6        percetageLostReplicas := (float64(numLostReplicas) / pgTotalReplicas)
             * 100
7
8        if percetageLostReplicas > 0.0 && percetageLostReplicas < 50.0 {
9           goodPgs = append(goodPgs, pg)
10       } else if percetageLostReplicas >= 50.0 && percetageLostReplicas <
             100.0 {
11          warningPgs = append(warningPgs, pg)
12       } else if percetageLostReplicas >= 100.0 {
13          lostPgs = append(lostPgs, pg)
14       }
15    }
16
17    compromisedPgs := [][]string{goodPgs, warningPgs, lostPgs}
18    allCompromisedPgs := []string{}
19
20    for _, ele := range compromisedPgs {
21       allCompromisedPgs = append(allCompromisedPgs, ele...)
22    }
23
24    allPgInAllOsds:= utility.RmvDuplStr(GetAllPgInAllOsds(pgDumpOutput))
25    inHealthPgs = utility.Difference(allPgInAllOsds,allCompromisedPgs)
26
```

```
27    return inHealthPgs, goodPgs, warningPgs, lostPgs
28 }
```

As evident, the analyzed functions require a faultOsdSlice as an input parameter. The GetDistributionMap function returns as output the distributionMap for the bucket in input, making it possible to trace the cascading impact of any bucket failure within the hierarchy back to the corresponding OSD slice. This acquired faultOsdSlice can subsequently be an input parameter for the abovementioned functions.

Incremental Data Loss Assessment Functions

The following functions have been implemented to develop a process that calculates the incremental data loss risk, taking into account the faults of OSDs, buckets in the hierarchy, and network addresses:

```
1  func RiskCalculator(faultBucketOrRouter string, pgDumpOutput
       s.PgDumpOutputStruct, osdTreeOutput s.OsdTreeOutputStruct,
       osdDumpOutput s.OsdDumpOutputStruct) ([]int, error) {
2     var affectedOsds []string
3
4     if faultBucketOrRouter == "" { //if blank input
5
6        affectedOsds = []string{}
7
8     } else {
9        addressRegex, _ := regexp.Compile("([0-9]{1,3}.){3}[0-9]{1,3}")
10
11       if addressRegex.MatchString(faultBucketOrRouter) { //if name is
             router (address)
12
13          publicAddressOsdsMap := GetPublicAddressOsdsMap(osdDumpOutput)
14          affectedOsds = publicAddressOsdsMap[faultBucketOrRouter]
15
16       } else { //if name is bucket
17
18          faultBucket := GetNode(faultBucketOrRouter, osdTreeOutput)
19
20          if faultBucket.Type == "osd" { //if bucket is osd
21
22             affectedOsds = append(affectedOsds, faultBucketOrRouter)
23
```

```
24          } else { //if the bucket is one in the hierarchy from host to root
25
26             if reflect.DeepEqual(faultBucket, s.NodeStruct{}) {
27
28                fmt.Println("RiskCalculator: error no bucket with this name
                     ")
29                err := errors.New("error no bucket with this name")
30
31                return nil, err
32
33             } else {
34
35                bucketDistributionMap :=
                     GetDistributionMap(faultBucket.Type, pgDumpOutput,
                     osdTreeOutput)
36                affectedOsds =
                     bucketDistributionMap[faultBucketOrRouter].Osd.([]string)
37             }
38          }
39       }
40    }
41
42    affectedOsdIds := GetOsdIdFromOsdNames(affectedOsds)
43
44    return affectedOsdIds, nil
45 }
```

```
1  func GetPublicAddressOsdsMap(osdDumpOutput s.OsdDumpOutputStruct)
       map[string][]string {
2     osds := osdDumpOutput.Osds
3     publicAddressOsdsMap := make(map[string][]string)
4
5     for _, osd := range osds {
6        publicAddress := strings.Split(osd.PublicAddr, ":")[0]
7
8        if _, isPresent := publicAddressOsdsMap[publicAddress]; !isPresent {
9           publicAddressOsdsMap[publicAddress] = []string{}
10       }
11
12       publicAddressOsdsMap[publicAddress] =
            append(publicAddressOsdsMap[publicAddress],
```

```
        "osd."+strconv.Itoa(osd.Osd))
13    }
14
15    return publicAddressOsdsMap
16 }
```

The RiskCalculator function mentioned above accepts as inputs the data structures that represent the current state of the Ceph cluster, along with a string that can represent an OSD, bucket, or network address. This last input is differentiated in its possible values using a regular expression. When a network address is provided, the function GetPublicAddressOsdsMap returns a map where the network addresses serve as the keys, and a slice of OSDs within its subnet are the corresponding values. On the other hand, when a bucket is provided, the function GetDistributionMap returns a map where the buckets serve as the keys, and their distributions across the Ceph hierarchy are the corresponding values. It is possible to determine the affected slice of OSDs using one of these maps associated with the specific input scenario.

The IncrementalRiskCalculator function, instead, accepts as input a slice of fault OSDs, buckets, or network addresses. Each is passed as input to the RiskCalculator function, and the total affected OSDs are calculated. The function outputs a map with the PGIDs as the keys and the number of affected replicas as the corresponding values.

```
1  func IncrementalRiskCalculator(faultBucketOrRouter []string, pgDumpOutput
       s.PgDumpOutputStruct, osdTreeOutput s.OsdTreeOutputStruct,
       osdDumpOutput s.OsdDumpOutputStruct) (map[string]int, error) {
2     totalNumberOfAffectedReplicaMap := make(map[string]int)
3     totalAffectedOsdIds := []int{}
4
5     for _, faultBucketOrRouterName := range faultBucketOrRouter {
6        affectedOsdIds, err := RiskCalculator(faultBucketOrRouterName,
             pgDumpOutput, osdTreeOutput, osdDumpOutput)
7        if err != nil {
8           fmt.Println("IncrementalRiskCalculator: Error in RiskCalculator")
9           return nil, err
10       }
11       totalAffectedOsdIds = append(totalAffectedOsdIds, affectedOsdIds...)
12    }
13
14    //removing duplicate
15    totalAffectedOsdIds = utility.RemoveDuplicateInt(totalAffectedOsdIds)
```

```
16    fmt.Printf("Total Affected Osds -> %d\n", totalAffectedOsdIds)
17
18    pgNumberOfAffectedReplicaMap :=
          GetPgNumberOfAffectedReplicaMap(totalAffectedOsdIds, pgDumpOutput)
19
20    for pg, numReplicas := range pgNumberOfAffectedReplicaMap {
21       if _, isPresent := totalNumberOfAffectedReplicaMap[pg]; !isPresent {
22          totalNumberOfAffectedReplicaMap[pg] = numReplicas
23       } else {
24          totalNumberOfAffectedReplicaMap[pg] += numReplicas
25       }
26    }
27
28    return totalNumberOfAffectedReplicaMap, nil
29 }
```

The GetPoolDataLossProbability function was implemented to calculate the data loss probability associated with each pool and thus to obtain a service-oriented data loss perspective. This function uses the outputs of the GetPoolPgNumberMap to get a map with pool IDs as the keys and the number of PGs within each corresponding pool as the values. The GetLostAndWarningPgs, instead, outputs the lostPgsSlice and the warningPgsSlice. These are obtained by considering the specific type of replication policy applied to the pool, whether it is a replication or an erasure-coding scheme.

```
1  func GetPoolDataLossProbability(faultBucketOrRouter []string, pgDumpOutput
       s.PgDumpOutputStruct, osdTreeOutput s.OsdTreeOutputStruct,
       osdDumpOutput s.OsdDumpOutputStruct) (map[string]float64, error) {
2
3     poolDataLossProbabilityMap := make(map[string]float64)
4
5     //get the number of PGs inside each pool
6     poolPgNumberMap, err := GetPoolPgNumberMap(pgDumpOutput, osdDumpOutput)
7
8     if err != nil {
9        error := errors.New("something went wrong in getting PoolPgNumberMap")
10       return poolDataLossProbabilityMap, error
11    }
12
13    //extract Pools' values from GetLostAndWarningPgs
14    poolAffectedPgsNumberMap := make(map[string]int)
15    lostPgsSlice, warningPgsSlice, err :=
```

```
        GetLostAndWarningPgs(faultBucketOrRouter, pgDumpOutput,
        osdTreeOutput, osdDumpOutput)
16    if err != nil {
17       fmt.Println("GetPoolDataLossProbability: Error in
            GetLostAndWarningPgs")
18       return nil, err
19    }
20    lostPgsPoolSlice := ExtractPoolsFromPgSlice(lostPgsSlice)
21
22    for _, pg := range warningPgsSlice {
23       poolId := strings.Split(pg, ".")[0]
24
25       if _, isPresent := poolAffectedPgsNumberMap[poolId]; !isPresent {
26          poolAffectedPgsNumberMap[poolId] = 0
27       }
28       poolAffectedPgsNumberMap[poolId] += 1
29    }
30
31    pools := GetPools(pgDumpOutput)
32
33    for _, pool := range pools {
34
35       if utility.StringInSlice(pool, lostPgsPoolSlice) {
36          poolAffectedPgsNumberMap[pool] = poolPgNumberMap[pool]
37       }
38
39       poolDatalossProbability := float64(poolAffectedPgsNumberMap[pool]) /
            float64(poolPgNumberMap[pool])
40
41       poolDataLossProbabilityMap[pool] =
            utility.RoundFloat(poolDatalossProbability, 2)
42    }
43
44    return poolDataLossProbabilityMap, nil
45 }
```

```
1  func GetPoolPgNumberMap(pgDumpOutput s.PgDumpOutputStruct, osdDumpOutput
       s.OsdDumpOutputStruct) (map[string]int, error) {
2     poolPgNumberMap := make(map[string]int)
3
4     poolSlice := GetPools(pgDumpOutput)
```

```
5
6    for _, poolId := range poolSlice {
7       pId, err := strconv.Atoi(poolId)
8
9       if err != nil {
10         error := errors.New("something went wrong in converting pool_id")
11         return poolPgNumberMap, error
12      }
13
14      pool := GetPool(pId, osdDumpOutput)
15      pgp_num := pool.PgNumTarget
16
17      poolPgNumberMap[poolId] = pgp_num
18   }
19   return poolPgNumberMap, nil
20 }
```

```
1  func GetLostAndWarningPgs(faultBucketOrRouter []string, pgDumpOutput
       s.PgDumpOutputStruct, osdTreeOutput s.OsdTreeOutputStruct,
       osdDumpOutput s.OsdDumpOutputStruct) ([]string, []string, error) {
2     warningPgsSlice := []string{}
3     lostPgsSlice := []string{}
4
5     incrementalPgAffectedReplicaMap, err :=
          IncrementalRiskCalculator(faultBucketOrRouter, pgDumpOutput,
          osdTreeOutput, osdDumpOutput)
6     if err != nil {
7        fmt.Println("GetLostAndWarningPgs: Error in
             IncrementalRiskCalculator")
8        return nil, nil, err
9     }
10
11    for pg, numPgAffectedReplicas := range incrementalPgAffectedReplicaMap {
12       numPgTotalReplicas, k, m := GetOsdNumberForPg(pg, osdDumpOutput)
             //Equal to Pool Size, in other words is the number of OSDs in
             which is stored the PG
13
14       numPgRemainingReplicas := int(numPgTotalReplicas) -
             numPgAffectedReplicas
15
16       if k == 0 && m == 0 { //replicated pool
```

```
17          if numPgRemainingReplicas <= 1 {
18             if numPgRemainingReplicas == 0 {
19                lostPgsSlice = append(lostPgsSlice, pg)
20             }
21
22             if numPgRemainingReplicas == 1 {
23                warningPgsSlice = append(warningPgsSlice, pg)
24             }
25          }
26       } else { //erasure coded pool
27          if numPgRemainingReplicas <= k {
28             if numPgRemainingReplicas < k {
29                lostPgsSlice = append(lostPgsSlice, pg)
30             }
31
32             if numPgRemainingReplicas == k {
33                warningPgsSlice = append(warningPgsSlice, pg)
34             }
35          }
36       }
37    }
38
39    return lostPgsSlice, warningPgsSlice, nil
40 }
```

Observing line 12 in the GetLostAndWarningPgs, it is possible to note the direct correlation between the number of degraded replicas and the number of affected OSDs.

```
1  func GetOsdNumberForPg(givenPg string, osdDumpOutput
       s.OsdDumpOutputStruct) (float64, int, int) {
2
3     stringPoolId := strings.Split(givenPg, ".")[0]
4
5     poolId, err := strconv.Atoi(stringPoolId)
6     if err != nil {
7        log.Fatal(err)
8     }
9
10    pool := GetPool(poolId, osdDumpOutput)
11
12    var k int //number of data chunks
```

```
13    var m int //number of coding chunks
14
15    osdNumberForPg := pool.Size
16
17    if pool.ErasureCodeProfile == "" {
18       //replicated pool
19       k = 0
20       m = 0
21    } else {
22       //erasure coded pool
23       k, m, err = GetErasureCodeProfileInfo(pool.ErasureCodeProfile,
             osdDumpOutput)
24
25       if err != nil {
26          return 0, 0, 0
27       }
28    }
29
30    return float64(osdNumberForPg), k, m
31 }
```

```
1  func GetErasureCodeProfileInfo(erasureCodeProfileName string, osdDumpOuput
       s.OsdDumpOutputStruct) (k int, m int, err error) {
2     erasureCodeProfilesStruct := osdDumpOuput.ErasureCodeProfiles
3     erasureCodeProfiles :=
          reflect.Indirect(reflect.ValueOf(&erasureCodeProfilesStruct))
4     numProfiles := erasureCodeProfiles.NumField()
5
6     for i := 0; i < numProfiles; i++ {
7        tag, ok := erasureCodeProfiles.Type().Field(i).Tag.Lookup("json")
8        if !ok {
9           error := errors.New("something went wrong in Tag.Lookup(\"json\")")
10          return 0, 0, error
11       }
12
13       erasureCodeProfileTag := utility.ExtractTagNameFromString(tag)
14
15       if erasureCodeProfileTag == erasureCodeProfileName {
16          erasureCodeProfile :=
               reflect.Indirect(erasureCodeProfiles).Field(i)
17          numDataChunk :=
```

```
            erasureCodeProfile.FieldByName("K").Interface().(string)
18          numCodingChunk :=
            erasureCodeProfile.FieldByName("M").Interface().(string)
19          k, err = strconv.Atoi(numDataChunk)
20          if err != nil {
21             error := errors.New("something went wrong in converting
                  numDataChunk to int")
22             return 0, 0, error
23          }
24          m, err = strconv.Atoi(numCodingChunk)
25          if err != nil {
26             error := errors.New("something went wrong in converting
                  numCodingChunk to int")
27             return 0, 0, error
28          }
29
30       }
31    }
32
33    return k, m, nil
34 }
```

In the case of a replicated pool, its PGs are included in the warningPgsSlice if their number of replicas after a fault equals 1. In this situation, the probability of data loss of the PGs is at its maximum because when the last remaining copy of a PG is affected, the result is data loss. At the same time, the replicated PGs are included in the lostPgsSlice when all their replicas are affected after a fault.

Considering an erasure-coded pool, instead, a PG within it is included in the warningPgsSlice if its number of remaining chunks after a fault equals k (number of data chunks). The reason is that a non-replaceable data chunk will be involved if an OSD containing the same PG is affected by a subsequent fault. At the same time, an erasure-coded PG is included in the lostPgsSlice when, after a fault, the number of the remaining chunks is less than k or all the chunks are affected.

Data Loss Forecasting Function

By evaluating the lifespan of storage devices, it is possible to make predictions regarding the future degradation of OSDs and, thus, their potential hardware faults. The RiskFailureForecasting function takes into input a slice of structs containing the lifetime info of the OSDs, and it is developed to provide the following output:

- A map with the OSD names as keys and the lifetime forecasting for the corresponding OSD as values
- A slice containing the OSDs whose lifetime is over 80%

The method by which the current OSD lifetime value is obtained exceeds the scope of this thesis.

```
type OsdLifetimeInfo struct {
   OsdName           string   `json:"osd_name"`
   InitiationDate    time.Time `json:"initiation_date"`
   CurrentOsdLifetime float64 `json:"current_osd_lifetime"`
}
```

```
func RiskFailureForecasting(osdLifetimeInfoStructs []s.OsdLifetimeInfo,
    givenTime time.Time) (osdLifeForecastingMap map[string]float64,
    warningOsdSlice []string) {

   currentTime := time.Now().UTC()
   timeHaed := givenTime.Sub(currentTime)
   timeHaedInWeeks := (timeHaed.Hours() / 24) / 7

   osdLifeForecastingMap = make(map[string]float64)

   for _, osdLifetimeInfo := range osdLifetimeInfoStructs {
      meanDegradationRate := GetOsdMeanDegradationRatePerWeek(currentTime,
          osdLifetimeInfo.InitiationDate,
          osdLifetimeInfo.CurrentOsdLifetime)

      osdLifeForecasting := osdLifetimeInfo.CurrentOsdLifetime +
          meanDegradationRate*timeHaedInWeeks

      if osdLifeForecasting > 100.0 {
         osdLifeForecasting = 100.0
      }

      osdLifeForecastingMap[osdLifetimeInfo.OsdName] =
          utility.RoundFloat(osdLifeForecasting, 2)
   }

   for key, value := range osdLifeForecastingMap {
      if value >= 80 {
         warningOsdSlice = append(warningOsdSlice, key)
```

```
        }
    }

    return
}
```

```
func GetOsdMeanDegradationRatePerWeek(currentTime time.Time,
    osdInitiationDate time.Time, currentOsdLifeTime float64)
    (meanDegradationRatePerWeek float64) {
    elapsedTime := currentTime.Sub(osdInitiationDate)
    elapsedWeeks := (elapsedTime.Hours() / 24) / 7

    meanDegradationRatePerWeek = currentOsdLifeTime / elapsedWeeks
    return
}
```

By utilizing the output (warningOsdSlice) obtained by the RiskFailureForecasting function as an input for the GetPoolDataLossProbability function, it becomes possible to generate a prediction regarding the progression of the data loss probability within the pools.

Service-Related Data Loss Evaluation System

A Service-Related Data Loss Evaluation System was developed to integrate the data loss probability and forecasting functions implemented in the preceding sections. This system consists of an RPC and REST API Server, a Reactive Client, and a Proactive one.

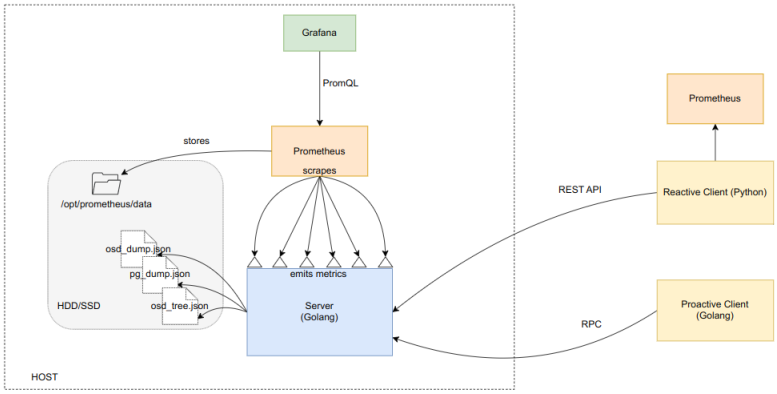


Figure 3.3: Service-Related Data Loss Evaluation System

RPC and REST API Server

The pool data loss probability and forecasting functions, developed in the preceding sections, provide their outcomes in response to REST API requests or RPCs. The Server component was designed to be deployed in a Ceph cluster and exposes the following REST (Representational State Transfer) endpoints for interaction:

- /health (GET)
  It was designed to verify the health of the server, and in a positive scenario, it responds with "System-Health: true."
- /dataloss-prob/faults (POST)
  Given the fault OSDs, buckets, or network addresses as request data, it outputs a JSON response containing the data loss probabilities of the pools.
- /dataloss-prob/forecasting (POST)
  Given the lifetime information of the OSDs in the cluster and specifying a timestamp for the forecasting date, it returns a JSON response containing a prediction for the lifetime of the OSDs up to the given date. Based on this prediction, the response also includes the data loss probabilities of the pools.
- /dataloss-prob/component/faults (POST)
- /dataloss-prob/component/forecasting (POST)

The last two endpoints respectively emit the current data loss probabilities of the pools and their prediction as Prometheus metrics for scraping. These will be discussed in detail in the Reactive Client subsection. The metrics emitted are the following:

- pool_dataloss_probability
- one_week_pool_dataloss_forecasting
- two_weeks_pool_dataloss_forecasting
- three_weeks_pool_dataloss_forecasting
- four_weeks_pool_dataloss_forecasting

The evaluation of data loss probabilities relies on the current state of the cluster, which is updated each time a request is received via the clusterStatsGathering function. This function exports and reads the content of the osd_dump, pg_dump, and osd_tree JSON files. The main endpoint handler implementations, categorized by type, are provided in the next page to keep things simple. The remaining portion of the code can be accessed and found on GitHub.

```go
func postFaultsProActive(w http.ResponseWriter, r *http.Request) {
   fmt.Println("REST Server: Post request arrived\n endpoint:
      /dataloss-prob/faults")

   w.Header().Set("Content-Type", "application/json")

   err := json.NewDecoder(r.Body).Decode(&faults)

   if err != nil {
      fmt.Println("Bad Request - There was an error in decoding the request
         body")

      response := "There was an error in decoding the request body"
      w.WriteHeader(http.StatusBadRequest)

      err = json.NewEncoder(w).Encode(&response)
      if err != nil {
         log.Fatalln("There was an error encoding the initialized struct")
      }
      return
   }

   for _, bucket := range faults {
      // Check over the acquired inputs
      faultsRegex, err := regexp.Compile(regexPattern)
      if err != nil {
         log.Fatalln("There was an error in Regex compile")
         return
      }

      if !faultsRegex.MatchString(bucket) {
         fmt.Println("Bad Request - Error in requested data")

         response := "Error in request data no correct bucket form"
         w.WriteHeader(http.StatusBadRequest)
         err = json.NewEncoder(w).Encode(&response)
         if err != nil {
            log.Fatalln("There was an error encoding the initialized
               struct")
         }
         return
```

```
39        }
40     }
41     //pg_dump, osd_dump, osd-tree
42     clusterStatsGathering()
43
44     //pool data loss probability calculation
45     poolDataLossProbability, err :=
          parser.GetPoolDataLossProbability(faults, pgDumpOutput,
          osdTreeOutput, osdDumpOutput)
46     if err != nil {
47        fmt.Println("Internal Server Error - Error calculating
             poolDataLossProbability - check if bucket name exists")
48
49        response := "Error in request data"
50        w.WriteHeader(http.StatusInternalServerError)
51
52        err = json.NewEncoder(w).Encode(&response)
53        if err != nil {
54           log.Fatalln("There was an error encoding the response")
55        }
56        return
57     }
58
59     response := FaultsProActiveResponse{
60        Faults:                faults,
61        PoolDatalossProbability: poolDataLossProbability,
62     }
63
64     w.WriteHeader(http.StatusOK)
65     err = json.NewEncoder(w).Encode(&response)
66     if err != nil {
67        log.Fatalln("There was an error encoding the response")
68     }
69  }
```

```
1  func postForecastingReActive(w http.ResponseWriter, r *http.Request) {
2     fmt.Println("REST Server: Post request arrived\n endpoint:
         /dataloss-prob/component/forecasting")
3
4     w.Header().Set("Content-Type", "application/json")
5
```

```
6    err := json.NewDecoder(r.Body).Decode(&osdLifetimeInfos)
7
8    if err != nil {
9       fmt.Println("Bad Request - There was an error in decoding the request
           body")
10
11      response := "There was an error in decoding the request body"
12      w.WriteHeader(http.StatusBadRequest)
13
14      err = json.NewEncoder(w).Encode(&response)
15      if err != nil {
16         log.Fatalln("There was an error encoding the initialized struct")
17      }
18      return
19   }
20
21   for _, info := range osdLifetimeInfos {
22      // Check on OsdName field
23      faultsRegex, err := regexp.Compile(regexPattern)
24      if err != nil {
25         log.Fatalln("There was an error in Regex compiling")
26         return
27      }
28
29      if !faultsRegex.MatchString(info.OsdName) {
30         fmt.Println("Bad Request - Error in requested data - osd_name")
31         response := "Error in request data - osd_name"
32         w.WriteHeader(http.StatusBadRequest)
33         err = json.NewEncoder(w).Encode(&response)
34         if err != nil {
35            log.Fatalln("There was an error encoding the initialized
               struct")
36         }
37         return
38      }
39   }
40   //pg_dump, osd_dump, osd-tree
41   clusterStatsGathering()
42
43   progressiveWeekDays := []int{7, 14, 21, 28}
44
```

```
45    for numWeek, numDays := range progressiveWeekDays {
46
47       fmt.Printf("Forecasting week n.%d\n", numWeek+1)
48
49       daysForwardTime := time.Now().AddDate(0, 0, numDays)
50
51       _, warningOsdSlice := parser.RiskFailureForecasting(osdLifetimeInfos,
            daysForwardTime)
52
53       //pool data loss probability calculation on WarningOsdSlice
54       poolDataLossProbability, err :=
            parser.GetPoolDataLossProbability(warningOsdSlice, pgDumpOutput,
            osdTreeOutput, osdDumpOutput)
55       if err != nil {
56          fmt.Println("Internal Server Error - Error calculating
               poolDataLossProbability - check if bucket name exists")
57
58          response := "Error in request data"
59          w.WriteHeader(http.StatusInternalServerError)
60          err = json.NewEncoder(w).Encode(&response)
61          if err != nil {
62             log.Fatalln("There was an error encoding the response")
63          }
64          return
65       }
66
67       pools := parser.GetPools(pgDumpOutput)
68
69       //Prometheus metric handling/triggering
70       switch numWeek {
71       case 0:
72          for _, pool := range pools {
73       oneWeekDataLossForecasting.With(prometheus.Labels{"pool": "Pool: " +
            pool}).Set(poolDataLossProbability[pool])
74          }
75       case 1:
76          for _, pool := range pools {
               twoWeeksDataLossForecasting.With(prometheus.Labels{"pool":
               "Pool: " + pool}).Set(poolDataLossProbability[pool])
77          }
78       case 2:
```

```
79          for _, pool := range pools {
80        threeWeeksDataLossForecasting.With(prometheus.Labels{"pool": "Pool: "
            + pool}).Set(poolDataLossProbability[pool])
81          }
82        case 3:
83          for _, pool := range pools {
              fourWeeksDataLossForecasting.With(prometheus.Labels{"pool":
              "Pool: " + pool}).Set(poolDataLossProbability[pool])
84          }
85        }
86    }
87    w.WriteHeader(http.StatusOK)
88    err = json.NewEncoder(w).Encode("Metrics
        '(one/two/three/four)WeeksDataLossForecasting' sent to Prometheus
        endpoint")
89    if err != nil {
90        log.Fatalln("There was an error encoding the initialized struct")
91    }
92  }
```

```
Started RPC server on localhost:1205
Listening ...
Started REST API server on localhost:8081
Listening ...
----------------------------------------
REST Server: Post request arrived
 endpoint: /dataloss-prob/faults
Total Affected Osds -> [1 2 4 6 8 12]
----------------------------------------
REST Server: Post request arrived
 endpoint: /dataloss-prob/forecasting
ForecastingTime-> 2025-06-12 02:53:00 +0000 UTC
OsdLifeForecastingMap-> map[osd.1:43.31 osd.2:100 osd.3:100]
WarningOsdSlice-> [osd.2 osd.3]
Total Affected Osds -> [2 3]
----------------------------------------
REST Server: Post request arrived
 endpoint: /dataloss-prob/component/faults
Total Affected Osds -> [1 8 6 2 4]
----------------------------------------
REST Server: Post request arrived
 endpoint: /dataloss-prob/component/forecasting
Forecasting week n.1
Total Affected Osds -> [2 3]
Forecasting week n.2
Total Affected Osds -> [2 3]
Forecasting week n.3
Total Affected Osds -> [2 3]
Forecasting week n.4
Total Affected Osds -> [2 3]
----------------------------------------
```

(a) Server outputs on received REST APIs

```
{faults:[osd.1,sv61,osd.8,10.22.22.3],pool_dataloss_pr
obability:{1:1,11:0.13,12:0.34,13:0.22,14:0.13,15:0.34
,19:0.28,2:0.22,21:1,27:0.31,28:0.22,29:0.19,3:1,30:0.
28,31:0.16,32:1,36:0.06,37:0.25,38:1,39:1,6:0.31,7:0.1
3,8:0.31}}
----------------------------------------
{osd_lifetime_forecasting:{osd.1:43.31,osd.2:100,osd.3
:100},warning_osds:[osd.2,osd.3],pool_dataloss_probabi
lity_forecasting:{1:0,11:0,12:0.03,13:0.03,14:0,15:0,1
9:0.09,2:0.03,21:1,27:0,28:0.03,29:0.03,3:1,30:0.03,31
:0.03,32:0.25,36:0,37:0,38:0.31,39:0.38,6:0.03,7:0,8:0
}}
----------------------------------------
PoolDataLossProbability metric sent to Prometheus endp
oint
----------------------------------------
Metrics (one/two/three/four)WeeksDataLossForecasting s
ent to Prometheus endpoint
----------------------------------------
```

(b) Server responses to REST APIs

Figure 3.4: Matching server and client output in a sequence of REST API requests

Furthermore, the server component can exchange information through RPCs (Remote Procedure Calls). Outlined below are the implementations of the RPC Server provided procedures:

```go
func (f *RpcServer) GiveFaults(args *Args, reply *RpcResponseStruct) error {
   fmt.Println("RPC Server: GiveFaults requested")

   //update pgDumpOutput, osdTreeOutput and osdDumpOutput
   clusterStatsGathering()

   for _, bucket := range args.Faults {
      // Check over the acquired inputs
      faultsRegex, err := regexp.Compile(regexPattern)
      if err != nil {
         return errors.New("error in regex compile")
      }

      if !faultsRegex.MatchString(bucket) {
         return errors.New("error in given faults %s")
      }

   }

   poolDataLossProbability, err := parser.GetPoolDataLossProbability(args.Faults, pgDumpOutput, osdTreeOutput, osdDumpOutput)
   if err != nil {
      return errors.New("error in data loss probability calculation - check if bucket name exists")
   }

   response := FaultsProActiveResponse{
      Faults:                 args.Faults,
      PoolDatalossProbability: poolDataLossProbability,
   }

   res := RpcResponseStruct{
      FaultsProActiveResponse:     response,
      OsdLifitimeForecastingResponse: OsdLifitimeForecastingResponse{}, //zero value
   }
```

```
34
35    *reply = res
36    return nil
37 }
```

```
1  func (f *RpcServer) GiveFaultsForecasting(args *Args1, reply
       *RpcResponseStruct) error {
2     fmt.Println("RPC Server: GiveFaultsForecasting requested")
3     //update pgDumpOutput, osdTreeOutput and osdDumpOutput
4     clusterStatsGathering()
5     osdLifetimeInfos := []structs.OsdLifetimeInfo{}
6
7     //each arg is a map[string]string
8     for _, arg := range args.OsdLifetimeInfos {
9        currOsdLifetime, err :=
             strconv.ParseFloat(arg["current_osd_lifetime"], 64)
10       if err != nil {
11          return errors.New("error in string to float64 conversion -
                currentOsdLifetime")
12       }
13       layout := "2006-01-02T15:04:05.000Z"
14       initiationTime, err := time.Parse(layout, arg["initiation_date"])
15       if err != nil {
16          return errors.New("error in string to time.Time conversion -
                initiation_date")
17       }
18
19       osdLifetimeInfo := structs.OsdLifetimeInfo{
20          OsdName:          arg["osd_name"],
21          InitiationDate:   initiationTime,
22          CurrentOsdLifetime: currOsdLifetime,
23       }
24
25       osdLifetimeInfos = append(osdLifetimeInfos, osdLifetimeInfo)
26    }
27
28    layout := "2006-01-02T15:04:05.000Z"
29    forecastingTime, err := time.Parse(layout, args.ForecastingTime)
30    if err != nil {
31       return errors.New("error in string to time.Time conversion -
             initiation_date")
```

```
32      }
33      fmt.Printf("forecastingTime-> %v\n", forecastingTime)
34      osdLifeForecastingMap, warningOsdSlice :=
            parser.RiskFailureForecasting(osdLifetimeInfos, forecastingTime)
35      fmt.Printf("osdLifeForecastingMap-> %v\nWarningOsdSlice-> %v\n",
            osdLifeForecastingMap, warningOsdSlice)
36
37      //pool data loss probability calculation on WarningOsdSlice
38      poolDataLossProbability, err :=
            parser.GetPoolDataLossProbability(warningOsdSlice, pgDumpOutput,
            osdTreeOutput, osdDumpOutput)
39      if err != nil {
40         return errors.New("error in data loss probability calculation - check
               if bucket name exists")
41      }
42      fmt.Printf("poolDataLossProbabilityMap-> %v\n", poolDataLossProbability)
43      response := OsdLifitimeForecastingResponse{
44         OsdLifetimeForecasting: osdLifeForecastingMap,
45         WarningOsds:           warningOsdSlice,
46         PoolDatalossProbability: poolDataLossProbability,
47      }
48      res := RpcResponseStruct{
49         FaultsProActiveResponse:     FaultsProActiveResponse{},
50         OsdLifitimeForecastingResponse: response, //zero value
51      }
52      *reply = res
53      return nil
54   }
```

Reactive Client

The Reactive Client was designed to be integrated into an ETL (Extract, Transform, Load) data pipeline workflow, periodically requesting information regarding the status of the OSDs from an existing Prometheus monitoring instance. Based on this information, the Reactive Client identifies the failed OSDs and forwards them to the server component for processing. The failed OSDs are transmitted as query data to the following endpoints:

- /dataloss-prob/component/faults (POST)
  Given the fault OSDs, buckets, or network addresses as request data, it outputs the data loss probabilities of the pools as Prometheus metric.

- /dataloss-prob/component/forecasting (POST)
  Given the lifetime information of the OSDs in the cluster as request data, it outputs a predictive forecast of the data loss probabilities for the pools, projecting them up to four weeks forward from the current time of the request. This forecasted data is then presented as a Prometheus metric.

Below is the code of the functions where the Reactive Client, written in Python, makes the REST requests. The remaining portion of the code can be accessed and found on GitHub.

```
def pool_data_loss_probability():
    try:
        #---extract num_osd_in from prometheus---
        if os.path.exists("prometheus_url.txt"):
            with open('prometheus_url.txt') as f:
                prometheus_url = f.readlines()[0].split('\n')[0]
        else:
            logging.warning("prometheus_url.txt not found")
        osd_in =requests.get(prometheus_url + '/api/v1/query',
            params={'query': 'ceph_osd_in'}).json() #returns a dictionary

        osds_in_prometheus = []
        if osd_in["status"] == "success":
            for result in osd_in["data"]["result"]:
                if result["value"][1] == "1":
                    osds_in_prometheus.append(result["metric"]["ceph_daemon"])
        else:
            logging.critical("Reading prometheus osd_in reported failure")
        osds_in_prometheus = set(osds_in_prometheus)
    # ... code ...
        if os.path.exists("osd_dump.json"):
            #parse osd_dump.json to find what osd are in
            with open("osd_dump.json") as osd_dump_file:
                osd_dump = json.load(osd_dump_file)
            osds_dump_in = []
            for osd in osd_dump["osds"]:
                if osd["in"]==1:
                    osds_dump_in.append("osd."+str(osd["osd"]))
            osds_dump_in=set(osds_dump_in)
    # ... code ...
                    # obtain osds_out to use in request
                    osds_out= set(osds_dump_in)-set(osds_in_prometheus)
```

```
                    osds_out=list(osds_out)
                    #send query
                    url =
                        'http://localhost:8081/dataloss-prob/component/faults'
                    response = requests.post(url, json= osds_out)
    # ... code ...
    except (requests.exceptions.JSONDecodeError,UnboundLocalError,
        json.decoder.JSONDecodeError, TypeError) as ex:
         logging.critical("Error in making request "+str(ex)+"\n")
```

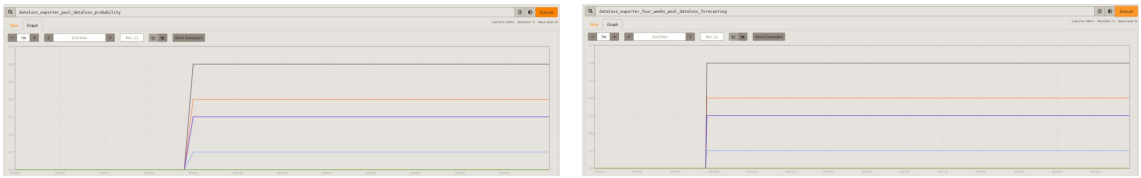
```
def pool_data_loss_forecasting():
    url = 'http://localhost:8081/dataloss-prob/component/forecasting'

    try:
        #Reading fake OSDs lifetime info
        with open ('osds_info_fake_data.json') as f:
            osd_info_forecasting = json.load(f)

        response = requests.post(url, json= osd_info_forecasting)
        print(response.text)

    except (requests.exceptions.JSONDecodeError,UnboundLocalError,
        json.decoder.JSONDecodeError, TypeError) as ex:
         logging.critical("Error in making request "+str(ex)+"\n")
```

An additional Prometheus instance later scrapes the metric values showing them reactively. Finally, a Grafana instance presents this information more appealingly within a dedicated dashboard.



(a) Pool Data Loss Probability (b) Four Weeks Forecasting

Figure 3.5: Prometheus metrics on the production environment

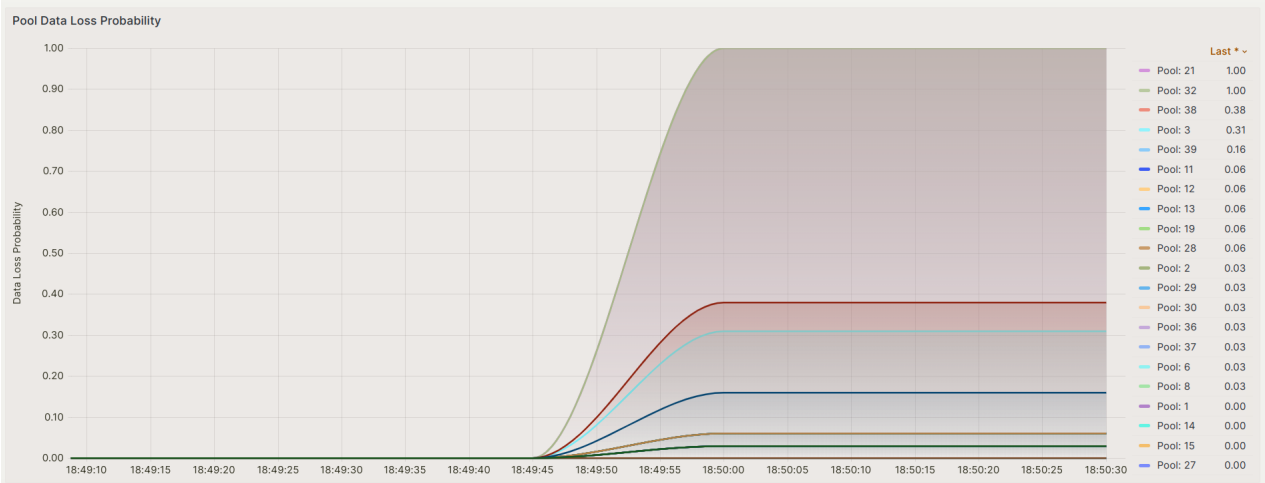


Figure 3.6: Pool Data Loss Probability graph (Grafana)

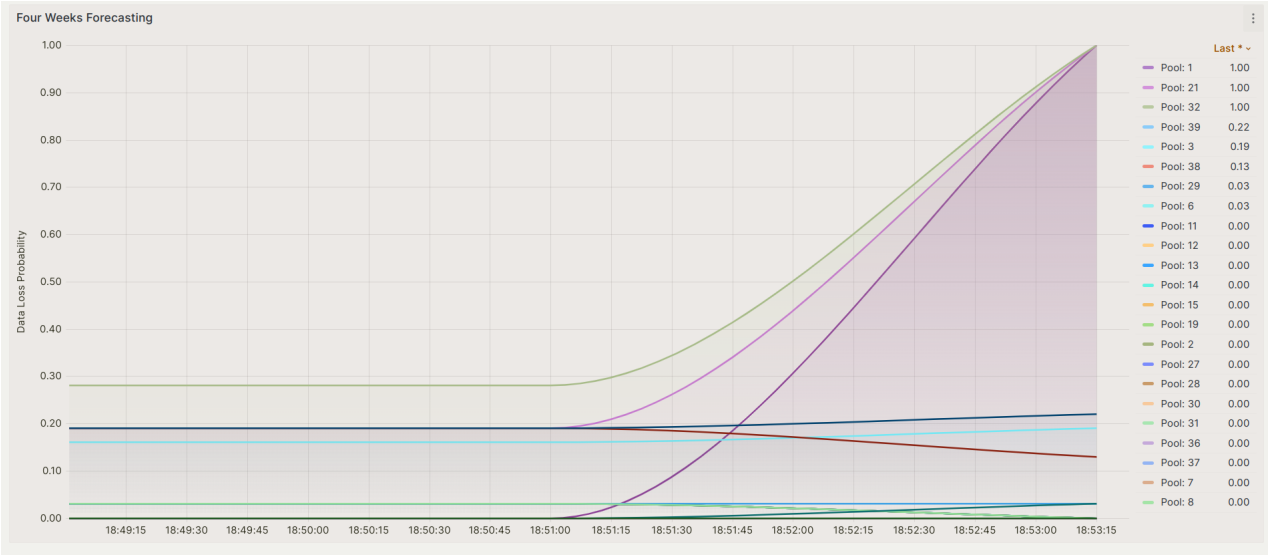


Figure 3.7: Four Weeks Forecasting graph (Grafana)

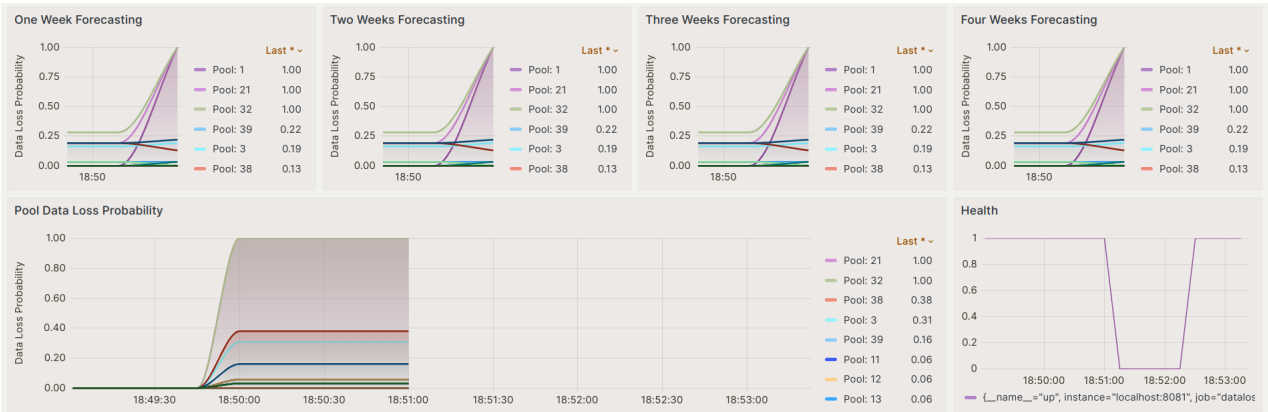


Figure 3.8: Grafana dashboard of the Service-Related Data Loss Evaluation System metrics

Proactive Client

The Proactive Client accepts command line arguments, with the first argument being either "faults" or "forecasting" to specify the desired mode. The second argument can be either a list of faults (in "faults" mode) or a timestamp indicating the desired prediction date (in "forecasting" mode). Below is the code section where the Proactive Client makes the remote procedure calls. The remaining portion of the code can be accessed and found on GitHub.

```
//      ...code...
//if first parameter equal "faults"

     // Invoke the remote function GiveFaults
     err = client.Call("RpcServer.GiveFaults", args, &reply)

     if err != nil {
        log.Fatal("Client invocation error: ", err)
     }
//if first parameter equal "forecasting"
//      ...code...
     // Invoke the remote function GiveFaultsForecasting
     err = client.Call("RpcServer.GiveFaultsForecasting", args, &reply)
     if err != nil {
        log.Fatal("Client invocation error: ", err)
     }
//      ...code...
```

The outputs obtained from the RPCs requested by the client are reported.

```
Started RPC server on localhost:1205
Listening ...
Started REST API server on localhost:8081
Listening ...
-----------------------------------------
RPC Server: GiveFaults requested
Total Affected Osds -> [1 5 2 4 6 12]
-----------------------------------------
RPC Server: GiveFaultsForecasting requested
forecastingTime-> 2025-06-12 02:53:00 +0000 UTC
osdLifeForecastingMap-> map[osd.1:93.97 osd.2:100]
WarningOsdSlice-> [osd.1 osd.2]
Total Affected Osds -> [1 2]
poolDataLossProbabilityMap-> map[1:0 11:0 12:0 13:0 14
:0 15:0 19:0 2:0 21:0.55 27:0 28:0 29:0 3:0.44 30:0 31
:0 32:0.5 36:0 37:0 38:0.38 39:0.56 6:0 7:0 8:0]
-----------------------------------------
```

(a) Server outputs on received RPCs

```
$ go run main.go faults osd.1,osd.5,10.22.22.3

2023/05/26 16:17:54 {faults:[osd.1,osd.5,10.22.22.3],p
ool_dataloss_probability:{1:0,11:0.66,12:0.66,13:0.63,
14:0.78,15:0.69,19:0.72,2:0.56,21:1,27:0.69,28:0.72,29
:0.63,3:1,30:0.63,31:0.69,32:1,36:0.84,37:0.66,38:1,39
:1,6:0.63,7:0.66,8:0.53}}
-----------------------------------------
$ go run main.go forecasting 2025-06-12T02:53:00.000Z

Forecasting Time Requested: 2025-06-12T02:53:00.000Z
2023/05/26 16:18:37 {osd_lifetime_forecasting:{osd.1:9
3.97,osd.2:100},warning_osds:[osd.1,osd.2],pool_datalo
ss_probability_forecasting:{1:0,11:0,12:0,13:0,14:0,15
:0,19:0,2:0,21:0.55,27:0,28:0,29:0,3:0.44,30:0,31:0,32
:0.5,36:0,37:0,38:0.38,39:0.56,6:0,7:0,8:0}}
```

(b) Server responses to RPCs

Figure 3.9: Matching server and client output in a sequence of RPCs

Conclusion

This thesis aimed to provide the Ceph administrator with a service-oriented view of data loss probability to help him choose and apply strategies to mitigate data loss. After understanding the Ceph architecture and delving into its issues requiring human intervention, the focus shifted to exploring data loss scenarios and the importance of data loss related to the pools, thus, to the services they support.

This study path was instrumental in understanding how complex the data loss problem is and how important it is to have a view of the data loss probability related to the service. Data loss depends on many factors, including the distribution of OSDs across buckets, the different redundancy strategies, and the various fault domain policies of pools. The storage device utilization rate and the network placement of OSDs further compound the complexity of this problem.
This complex, heterogeneous, and ever-changing scenario requires tools that the Ceph system administrator can deploy on a Ceph cluster to have a broader view of the data loss related to the pools and, thus, a comprehensive perspective on service-related data loss probabilities.

To achieve this objective, the Service-Related Data Loss Evaluation System was implemented. This system proactively and reactively gathers information from a Ceph cluster and, based on them, can provide service-related data loss probabilities and their forecasting up to a given date or up to four weeks forward from the current time of a request. Furthermore, these pieces of information are scraped by Prometheus and shown in a Grafana dashboard to provide the Ceph system administrator with a view of the data loss probabilities of the pools in the cluster.

Possible future developments include further explorations on data loss scenarios and methodologies for accurately determining the lifespan of various storage devices. These developments would further enhance the assessment and forecasting processes.

Bibliography

[1] By Apache Airflow. *Concepts - Core Ideas*. 2023. URL: https://airflow.apache.org/docs/apache-airflow/1.10.10/concepts.html (visited on 03/07/2023).

[2] By Ignacio Barahona. *What is unstructured data?* 2022. URL: https://benelux.nttdata.com/insights/blog/what-is-unstructured-data (visited on 04/27/2023).

[3] By Stacey Burbach. *Block Storage vs. Object Storage vs. File Storage: What's the Difference?* 2022. URL: https://qumulo.com/blog/block-storage-vs-object-storage-vs-file-storage/ (visited on 04/18/2023).

[4] Tony Piscopo By Yadin Porter de León and Druva Marketing. *Object Storage versus Block Storage: Understanding the Technology Differences*. 2020. URL: https://www.druva.com/blog/object-storage-versus-block-storage-understanding-technology-differences (visited on 04/18/2023).

[5] By Cedric and Jean. *SSD vs HDD Lifespan, Which Lasts Longer SSD or HDD? What's the Difference*. 2023. URL: https://www.easeus.com/storage-media-recovery/ssd-vs-hdd-lifespan.html#:~:text=While%20HDDs%20tend%20to%20last,to%2010%20years%20or%20more (visited on 05/10/2023).

[6] By Docs Ceph. *CEPH BLOCK DEVICE*. 2023. URL: https://docs.ceph.com/en/quincy/rbd/index.html (visited on 05/01/2023).

[7] By Docs Ceph. *CEPH OBJECT GATEWAY*. 2023. URL: https://docs.ceph.com/en/latest/radosgw/ (visited on 05/01/2023).

[8] By Docs Ceph. *CEPH STORAGE CLUSTER and ARCHITECTURE*. 2023. URL: https://docs.ceph.com/en/latest/architecture/ (visited on 03/10/2023).

[9] By Docs Ceph. *MONITORING OSDS AND PGS*. 2023. URL: https://docs.ceph.com/en/quincy/rados/operations/monitoring-osd-pg/ (visited on 05/09/2023).

[10] By Docs Ceph. *STORAGE DEVICES*. 2023. URL: https://docs.ceph.com/en/latest/rados/configuration/storage-devices/#:~:text=Ceph%20OSDs%20(Object%20Storage%20Daemons,solid%20state%20disk%20(SSD) (visited on 03/10/2023).

[11] By Docs Ceph. *TROUBLESHOOTING MONITORS*. 2023. URL: https://docs.ceph.com/en/quincy/rados/troubleshooting/troubleshooting-mon/ (visited on 05/07/2023).

[12] By Docs Ceph. *TROUBLESHOOTING OSDS*. 2023. URL: https://docs.ceph.com/en/quincy/rados/troubleshooting/troubleshooting-osd/ (visited on 05/08/2023).

[13] By Docs Ceph. *TROUBLESHOOTING PGS*. 2023. URL: https://docs.ceph.com/en/quincy/rados/troubleshooting/troubleshooting-pg/ (visited on 05/09/2023).

[14] By Cloudian. *Object Storage vs. File Storage: What's the Difference?* URL: https://www.druva.com/blog/object-storage-versus-block-storage-understanding-technology-differences (visited on 04/18/2023).

[15] By Jency Durairaj. *2.5 Quintillion Bytes of Data Generated Everyday - Top Data Science Trends 2020*. 2020. URL: https://us.sganalytics.com/blog/2-5-quintillion-bytes-of-data-generated-everyday-top-data-science-trends-2020/ (visited on 04/26/2023).

[16] By Google. *Google Trends: big data*. 2023. URL: https://trends.google.it/trends/explore?date=all&q=big%20data&hl=it (visited on 04/26/2023).

[17] Red Hat. *Red Hat Ceph Storage 5 Guide on Red Hat Ceph Storage Architecture*. 2022. URL: https://access.redhat.com/documentation/it-it/red_hat_ceph_storage/5/html-single/architecture_guide/index (visited on 02/15/2023).

[18] By Bojan Jovanovic. *Internet of Things statistics for 2023 – Taking Things Apart*. 2023. URL: https://dataprot.net/statistics/iot-statistics/#:~:text=drastically%20improve%20efficiency.-,The%20amount%20of%20data%20generated%20by%20IoT%20devices%20is%20expected,ZB%20(zettabytes)%20by%202025.&text=IoT%20big%20data%20statistics%20show,data%20in%20the%20following%20years (visited on 04/26/2023).

[19] By Kevin Kelly. *Scan This Book!* 2006. URL: https://www.nytimes.com/2006/05/14/magazine/14publishing.html (visited on 04/25/2023).

[20] By Jason Kincaid. *Google's Go: A New Programming Language That's Python Meets C++*. 2009. URL: https://shorturl.at/flyEH (visited on 03/10/2023).

[21] By Jason Kincaid. *rpc - package - standard library (go1.20.4)*. 2023. URL: https://pkg.go.dev/net/rpc (visited on 03/11/2023).

[22] "Dave Kuhlman". *"A Python Book: Beginning Python, Advanced Python, and Python Exercises"*. "dkuhlman@rexx.com http://www.rexx.com/ dkuhlman", 2012.

[23] By Sebastian Lambert. *Number of Internet Users in 2022/2023: Statistics, Current Trends, and Predictions*. 2023. URL: https://financesonline.com/number-of-internet-users/ (visited on 04/26/2023).

[24] Karan Singh Michael Hackett Vikhyat Umrao. *Ceph designing and implementing scalable storage*. Packt, 2019.

[25] By Philippe Nicolas. *The History Boys: Object storage ... from the beginning*. 2016. URL: https://www.theregister.com/2016/07/15/the_history_boys_cas_and_object_storage_map/ (visited on 04/18/2023).

[26] By Prometheus. *Prometheus - METRIC TYPES*. 2023. URL: https://prometheus.io/docs/concepts/metric_types/ (visited on 04/01/2023).

[27] By Prometheus. *Prometheus - Overview*. 2023. URL: https://prometheus.io/docs/introduction/overview/ (visited on 04/01/2023).

[28] By Likhith Shankar. *Data Generated by a Person Every Second*. 2020. URL: https://medium.com/@likhithshankarprithvi/data-generated-by-a-person-per-second-b14a8cb60572 (visited on 04/25/2023).

[29] By Carol Sliwa. *RADOS Block Device (RBD)*. 2014. URL: https://www.techtarget.com/searchstorage/definition/RADOS-Block-Device-RBD#:~:text=A%20RADOS%20Block%20Device%20 (visited on 04/16/2023).

[30] Dan van der Ster (CERN). *Ceph Storage at CERN*. 2022. URL: https://indico.gsi.de/event/14500/ (visited on 04/07/2023).

[31] Vinodh Venkatesan et al. "Reliability of Clustered vs. Declustered Replica Placement in Data Storage Systems". In: *2011 IEEE 19th Annual International Symposium on Modelling, Analysis, and Simulation of Computer and Telecommunication Systems*. 2011, pp. 307–317. DOI: 10.1109/MASCOTS.2011.53.

[32] By Kashyap Vyas. *How Long Does an SSD Last? | Calculate Your SSD's Lifespan*. 2023. URL: https://www.enterprisestorageforum.com/hardware/ssd-lifespan-how-long-will-your-ssd-work/ (visited on 05/10/2023).

[33] Sage Weil et al. "Ceph: A Scalable, High-Performance Distributed File System." In: Nov. 2006, pp. 307–320.

[34] Sage Weil et al. "RADOS: A scalable, reliable storage service for petabyte-scale storage clusters". In: Nov. 2007, pp. 35–44. DOI: 10.1145/1374596.1374606.

[35] Sage A. Weil et al. "CRUSH: Controlled, Scalable, Decentralized Placement of Replicated Data". In: *SC '06: Proceedings of the 2006 ACM/IEEE Conference on Supercomputing*. 2006, pp. 31–31. DOI: 10.1109/SC.2006.19.

[36] By Wikipedia. *Grafana*. 2023. URL: https://en.wikipedia.org/wiki/Grafana (visited on 04/03/2023).